Introduction to Software Engineering

Software Design



Software Engineering Department
Faculty of Information and Technology
University of Science

Contents

Objectives 4

1 Member Contribution Assessment 4

2 Design Overview 5

2.1 Design Goals and Constraints 5

3 Architectural Design 9

3.1 Architectural Style 9

3.2 System Context Diagram (C4 level 1) 10

3.3 Container Diagram (C4 level 2) 10

3.4 Component Diagram 10

3.5 Deployment Diagram 10

3.6 Architecture Decision Records 10

4 Object-Oriented Design 11

4.1 Domain Model 11

4.2 Class Diagram 16

4.3 Class Specifications 23

4.4 Sequence Diagrams 32

4.5 State Machine Diagrams 41

4.6 Design Principles and Patterns 44

5 Data Design 46

5.1 Data Diagram 46

5.2 Data Specification 46

5.3 Normalisation and Indexes 46

6 User Interface and User Experience Design 46

6.1 Screen List and Screen Flow 46
6.2 Screen Specifications 47
6.3 Accessibility Checklist 47

7 Traceability Update **48**

8 Changes to the Requirements Since PA2 **48**

9 AI Usage Log **48**

Software Design

Objectives

This document focus on the following topics:

- ✓ Completing the Software Design document with the following sections:
  - Architectural design (C4, ADR, deployment)
  - Object-oriented design (domain model, class and sequence diagrams)
  - Data design
  - User interface design
  - Traceability update
- ✓ Including an AI usage log for this phase
- ✓ Understanding the Software Design document.

1 Member Contribution Assessment

The contribution percentages and assigned design modules for the Software Design phase (PA3) were agreed by the team. They total 100%. Evidence is taken from the team's GitHub issues, commits, and merged pull requests.

ID	Name	Role(s)	Contribution (%)	Evidence
24127434	Đỗ Vĩnh Kỳ	Team Lead / PM	20	Overall design integration, architectural style, and C4 models review; repository commits.

24127374	Huỳnh Thái Hòa	Business Analyst	20	Section 2.1 (Design Goals, Priority FRs/NFRs, Constraints), Section 4.1 (Domain Model, TikZ diagram, Business Rules Verification), and Handover documentation; branch feature/pa3-sections-2.1-4.1.
24127117	Nguyễn Anh Tá	Designer / Architect	20	Sections 4.2--4.6 (Class, Sequence, State Machine diagrams) and Section 6 (UI/UX Design).
24127357	Nguyễn Gia	Developer	20	Section 3 (ADRs, Deployment diagram) and Section 5 (Data Design, Schema, Normalisation).
24127490	Trần Thiên Phú	Tester	20	Section 7 (Traceability Update) and Section 8 (Changes to Requirements since PA2).

2 Design Overview

2.1 Design Goals and Constraints

Thiết kế kiến trúc và mô hình hóa hệ thống US-Recovery (CampusHub) được xây dựng dựa trên việc thỏa mãn các Yêu cầu có ý nghĩa kiến trúc (Architecturally Significant Requirements -- ASRs). Các yêu cầu này bao gồm các tính năng nghiệp vụ cốt lõi (Core Functional Drivers), các thuộc tính chất lượng then chốt (Key Quality Attributes / NFRs), cùng hệ thống các ràng buộc kỹ thuật, ngân sách và tiến độ khắt khe của dự án môn học Nhập môn Công nghệ Phần mềm.

2.1.1 Architecturally Significant Functional Requirements

Các yêu cầu chức năng có mức ưu tiên cao nhất (Must-have) được xác định từ PA1 và đặc tả chi tiết trong PA2 đóng vai trò then chốt trong việc định hình các thành phần kiến trúc, mô hình dữ liệu và cơ chế bảo mật của hệ thống:

ID	Tính năng (Feature)	Yêu cầu chức năng cốt lõi	Tác động định hướng thiết kế

FR-01	Xác thực tài khoản (@...edu.vn)	Bắt buộc đăng ký và đăng nhập thông qua email chính danh trường đại học. Cung cấp mã OTP/link kích hoạt để định danh sinh viên.	Thiết kế module xác thực (Auth Service) tích hợp Supabase Auth và SMTP Service; phát hành JWT token để xác thực phiên làm việc không trạng thái (stateless).
FR-02	Phân quyền người dùng (RBAC)	Phân định rạch ròi 3 vai trò: Khách (Guest -- chỉ xem/tìm kiếm), Sinh viên (Student -- đăng bài, claim, quản lý tin) và Quản trị viên (Admin -- kiểm duyệt, cấu hình).	Cài đặt Middleware kiểm soát truy cập phân tầng (Role-based Authorization Middleware) tại Backend và cơ chế Route Guards bảo vệ các trang điều hướng tại Frontend.
FR-03	Quản lý bài đăng Mất/Nhặt đồ	Cho phép sinh viên tạo, sửa, đóng bài đăng; đính kèm ảnh thực tế, chọn cơ sở trường, phân loại danh mục, thời gian và địa điểm chi tiết.	Thiết kế mô hình dữ liệu tách biệt giữa CSDL quan hệ (PostgreSQL lưu metadata) và Cloud Storage (lưu trữ tệp ảnh tĩnh); xây dựng máy trạng thái bài đăng (Active, Resolved, Closed).
FR-04	Tìm kiếm & Lọc đa chiều	Cho phép tìm kiếm toàn văn theo từ khóa và bộ lọc kết hợp: Cơ sở trường, Danh mục vật phẩm, Khoảng thời gian, Loại bài đăng (Mất/Nhặt).	Thiết kế chỉ mục tìm kiếm tối ưu (PostgreSQL B-Tree trên các trường lọc, GIN index trên tiêu đề/nội dung), xử lý phân trang dữ liệu (Pagination) giảm tải mạng.
FR-05 FR-06	Quy trình nhận đồ (Claim Workflow)	Sinh viên gửi yêu cầu Claim đến bài Nhặt đồ hoặc bài Mất đồ với thông tin xác minh phù hợp từng loại bài. Chủ bài duyệt (Accept/Reject) để mở khóa thông tin liên hệ bảo mật.	Thiết kế luồng xử lý trạng thái Claim (State Pattern), cơ chế kiểm duyệt hai bước (Two-step verification), và che giấu dữ liệu nhạy cảm (Data Masking) tại tầng API.
FR-07 FR-08	Kiểm duyệt & Quản trị nền tảng	Bảng điều khiển Admin xử lý báo cáo bài đăng vi phạm (ngưỡng $\geq$ 3 báo cáo theo BR-03 sẽ tự động chuyển vào hàng đợi), khóa tài khoản, duyệt cơ sở mới.	Thiết kế kiến trúc hàng đợi xử lý báo cáo (Moderation Queue), trigger cập nhật trạng thái tự động và các endpoint quản trị chuyên biệt được bảo vệ nghiêm ngặt.
FR-09	Thông báo nội bộ (In-app Notification)	Tự động gửi thông báo cho người dùng khi có tương tác Claim mới, thay đổi trạng thái bài đăng hoặc cảnh báo từ quản trị viên.	Thiết kế phân hệ lưu trữ và phân phối thông báo (Notification Subsystem), hỗ trợ truy vấn danh sách thông báo chưa đọc theo thời gian thực.

2.1.2 Key Quality Attributes and Non-Functional Requirements

Hệ thống phải đáp ứng các tiêu chuẩn chất lượng (NFRs) đã cam kết trong PA2, qua đó chi phối trực tiếp đến các giải pháp kiến trúc:

ID	Thuộc tính (Quality)	Chỉ số đo lường (Measurable Metric)	Giải pháp thiết kế kiến trúc
NFR-01	Performance (Hiệu năng)	Dưới tải 100 người dùng đồng thời, thời gian phản hồi tìm kiếm/lọc phải đạt P95 $\leq$ 2 giây.	Thiết kế kiến trúc phi trạng thái (Stateless REST API); thiết lập chỉ mục (Indexes) trên các khóa ngoại và cột lọc; giới hạn kích thước phân trang (Page Size); nén ảnh phía client trước khi upload.
NFR-02	Security & Privacy (Bảo mật & Riêng tư)	100% kết nối sử dụng giao thức HTTPS. Tuyệt đối không để lộ thông tin liên hệ (SĐT, Email cá nhân) của chủ bài đăng cho tới khi Claim được chấp thuận (BR-02).	Triển khai cơ chế Data Masking tại Business Service; áp dụng nguyên tắc đặc quyền tối thiểu (Least Privilege); cấu hình Row Level Security (RLS) trên PostgreSQL; băm mật khẩu bằng thuật toán bcrypt.
NFR-03	Usability (Tính khả dụng)	Sinh viên mới có thể hoàn tất việc tạo một bài đăng hoàn chỉnh trong vòng $\leq$ 3 phút mà không cần tài liệu hướng dẫn.	Thiết kế luồng giao diện tinh gọn, trực quan (Wizard/Modal form); tự động điền thông tin người dùng từ token; cung cấp phản hồi lỗi trực quan ngay tại trường nhập liệu (Client-side validation).
NFR-04	Compatibility (Khả năng tương thích)	Giao diện thích ứng (Responsive) hoàn hảo trên các trình duyệt Chrome, Firefox, Safari từ màn hình di động (360px) đến máy tính để bàn (1920px), không bị vỡ bố cục hoặc cuộn ngang.	Sử dụng kiến trúc Single Page Application (SPA) với React và Tailwind CSS theo triết lý Mobile-First; xây dựng các UI component linh hoạt (Flexbox, CSS Grid).
NFR-05	Availability (Tính sẵn sàng)	Hệ thống duy trì tính sẵn sàng (uptime) $\geq$ 99% trong suốt giai đoạn thử nghiệm và đánh giá nghiệm thu.	Tận dụng hạ tầng đám mây PaaS/BaaS (Vercel, Render) có cơ chế auto-restart khi gặp sự cố; thiết lập giám sát định kỳ thông qua UptimeRobot; tách biệt database độc lập với application server.

2.1.3 Verification of Core Business Rules

Hệ thống thiết kế phải đảm bảo thực thi triệt để 4 quy tắc nghiệp vụ cốt lõi đã cam kết từ PA2, đóng vai trò là các chốt chặn kiến trúc (Architectural Invariants):

Quy tắc	Nội dung quy tắc	Cơ chế kiến trúc bảo đảm	Tiêu chí xác minh (Verification)
BR-01	Xác thực Email trường (@...edu.vn)	Chỉ những tài khoản đã xác minh email chính danh trường đại học mới được phép tạo bài đăng và gửi Claim.	Auth Middleware kiểm tra cờ `is_verified = true` trước khi cho phép gửi request tạo Post hoặc Claim. Khách chỉ có quyền đọc/tìm kiếm (Read-only).
BR-02	Bảo vệ thông tin liên hệ (Protected Contact)	Số điện thoại và email cá nhân của chủ bài đăng tuyệt đối không hiển thị công khai; chỉ được mở khóa khi Claim được chấp thuận.	Cơ chế Data Masking tại tầng Business Service. API chi tiết bài đăng mặc định loại bỏ trường thông tin liên hệ trừ khi caller là chủ sở hữu hoặc Claimant đã được duyệt.
BR-03	Ngưỡng kiểm duyệt (Moderation Threshold)	Bất kỳ bài đăng nào nhận từ 3 báo cáo vi phạm trở lên ($\geq$ 3 reports) sẽ tự động bị ẩn khỏi danh sách công khai và chuyển vào hàng đợi Admin. Bài chỉ được hiển thị lại nếu Admin quyết định giữ bài.	Database Trigger / Service Hook đếm số lượng bản ghi Report hợp lệ. Khi đạt ngưỡng 3, tự động chuyển `status` của Post sang `REPORTED`, ẩn khỏi danh sách công khai và gửi thông báo cảnh báo.
BR-04	Quy trình duyệt Claim (Claim Approval)	Claim có thể được gửi đến bài Nhặt đồ hoặc bài Mất đồ. Chủ bài đánh giá thông tin xác minh theo loại bài để duyệt (Accept) hoặc từ chối (Reject) Claim. Khi duyệt, bài đăng tự động hoàn tất.	Thiết kế máy trạng thái Claim hữu hạn (`PENDING` $\rightarrow$ `ACCEPTED` / `REJECTED`). Sử dụng Database Transaction để đồng thời cập nhật Claim sang `ACCEPTED` và Post sang `RESOLVED`.

2.1.4 Design and Implementation Constraints

Thiết kế phần mềm phải tuân thủ nghiêm ngặt các giới hạn về công nghệ, tài chính và tiến độ:

1. **Ràng buộc Công nghệ (Technology Constraints):**
   - **Frontend:** Ứng dụng nền tảng web React 18+ kết hợp Vite làm công cụ build tối ưu, sử dụng React Router cho điều hướng SPA và Tailwind CSS để xây dựng hệ thống design tokens responsive.
   - **Backend:** Sử dụng Node.js (LTS) với nền tảng Express.js để xây dựng RESTful Web APIs chuẩn JSON. Cấu trúc mã nguồn tuân thủ nghiêm ngặt kiến trúc phân tầng (Layered Architecture: Routing -- Controller -- Service -- Data Access).

- **Cơ sở dữ liệu & Backend Services:** Sử dụng dịch vụ Supabase trên nền PostgreSQL 15+, tận dụng cơ chế Supabase Auth cho quản lý phiên và Supabase Storage làm kho lưu trữ hình ảnh vật phẩm.
- **Dịch vụ bên thứ ba:** Tích hợp Brevo SMTP Server để gửi email kích hoạt và mã xác thực OTP chính danh sinh viên.
- **Triển khai (Deployment):** Mã nguồn Frontend triển khai tự động trên Vercel CDN; Backend triển khai dưới dạng web service trên nền tảng Render (máy chủ đặt tại khu vực Singapore để tối ưu độ trễ mạng đối với người dùng tại TP.HCM).

2. **Ràng buộc Ngân sách (Budget Constraints):**
   - **Chi phí triển khai 0 VNĐ (Zero-Budget):** Dự án vận hành hoàn toàn dựa trên các gói tài nguyên miễn phí (Free Tiers) của các nền tảng đám mây (Vercel Hobby, Render Free Web Service, Supabase Free Project, Brevo Free 300 emails/day).
   - **Tối ưu hóa tài nguyên phần cứng:** Do gói máy chủ Render Free bị giới hạn RAM ở mức 512MB và có hiện tượng sleep sau thời gian nhàn rỗi, kiến trúc Backend phải thiết kế gọn nhẹ (lightweight), không chạy các tác vụ chạy nền (background workers) tốn bộ nhớ, đồng thời phía Frontend cần có thông báo chờ tải (skeleton loading/spinner) thân thiện.
3. **Ràng buộc Tiến độ và Quy trình (Schedule and Process Constraints):**
   - **Phương pháp luận phát triển:** Tuân thủ mô hình Waterfall theo đúng đề cương môn học, chia làm 5 mốc nghiệm thu tương ứng với các đồ án thành phần (PA1 đến PA5). Thiết kế tại PA3 đóng vai trò là bản thiết kế khung (Blueprint) bất biến cho giai đoạn cài đặt.
   - **Thời hạn lập trình (Coding Window):** Giai đoạn hiện thực mã nguồn (PA5) bị giới hạn nghiêm ngặt trong tối đa **3 tuần** làm việc của nhóm 5 thành viên. Do đó, thiết kế hệ thống phải tập trung cao độ vào tính khả thi, hạn chế thiết kế dư thừa (over-engineering), ưu tiên hoàn thiện xuất sắc 100% các tính năng cốt lõi (Must-have).
   - **Quy chuẩn làm việc nhóm:** Quản lý mã nguồn tập trung trên GitHub; 100% commit phải tuân thủ chuẩn Conventional Commits; bắt buộc tạo Pull Request và trải qua quy trình Peer Review (ít nhất 1 thành viên kiểm tra chéo) trước khi tích hợp vào nhánh chính (`main`).

3 Architectural Design

3.1 Architectural Style

[Name the architectural style(s) used (layered, MVC, client--server, microservices, event-driven, ...) and justify the choice against the design goals.]

3.2 System Context Diagram (C4 level 1)

[Update the context diagram from PA1.]

3.3 Container Diagram (C4 level 2)

[Show web/mobile apps, API/back end, database, external services and the technology of each container.]

3.4 Component Diagram

[Decompose the main container(s) into components/modules and show their dependencies (system decomposition).]

3.5 Deployment Diagram

[Show where each component runs (devices, servers/cloud services, database server) and how they communicate.]

3.6 Architecture Decision Records

[Record at least 3 important decisions (e.g., database, framework, authentication, hosting). One table per ADR; store them in `/docs/design/adr`.]

ADR	ADR-001: <title>
Status	proposed | accepted | superseded
Context	
Decision	
Alternatives considered	

Consequences	

4 Object-Oriented Design

4.1 Domain Model

Mô hình lĩnh vực (Domain Model) biểu diễn các thực thể khái niệm (Conceptual Business Entities) và mối quan hệ ngữ nghĩa thế giới thực trong hệ thống US-Recovery. Tuân thủ nguyên lý phân tích hướng đối tượng (OOA), Domain Model tập trung vào việc làm rõ bản chất nghiệp vụ, ý nghĩa thuộc tính và các quy tắc bất biến, **hoàn toàn không chứa phương thức xử lý (no methods)** cũng như không phụ thuộc vào chi tiết cài đặt kỹ thuật (khóa ngoại, chỉ mục, REST controller hay giao diện).

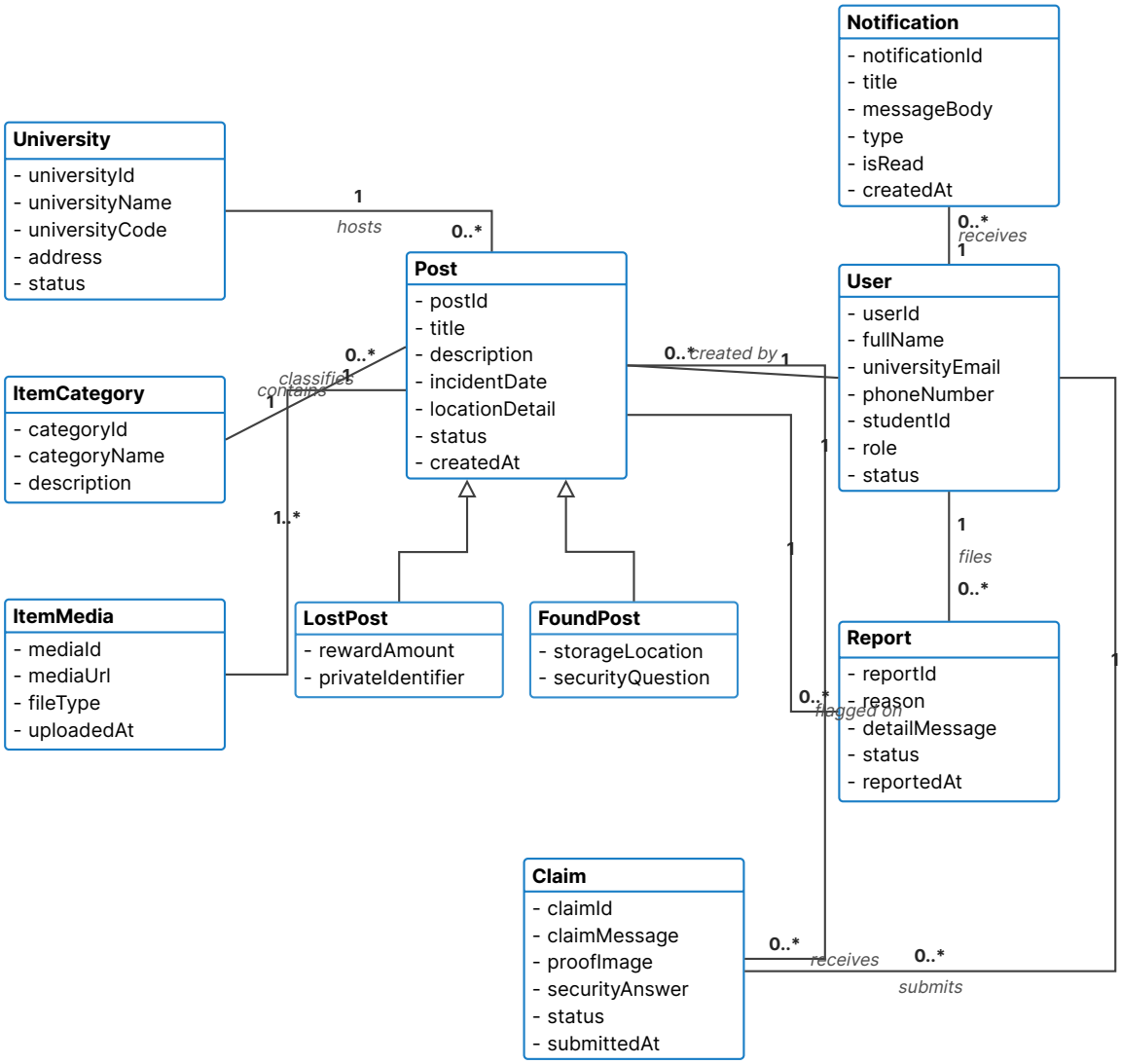


Figure 1: Domain Model diagram for US-Recovery

4.1.1 Domain Entities Specification

Bảng dưới đây đặc tả chi tiết ý nghĩa nghiệp vụ và các thuộc tính khái niệm của từng thực thể trong miền bài toán:

Thực thể	Định nghĩa nghiệp vụ	Thuộc tính khái niệm	Quy tắc ràng buộc nghiệp vụ

User	Đại diện cho người dùng đã định danh trong trường đại học hoặc quản trị viên hệ thống.	userId, fullName, universityEmail, phoneNumber, studentId, role (GUEST, STUDENT, ADMIN), status (ACTIVE, BANNED)	Bắt buộc phải có email đuôi @...edu.vn đã xác thực (BR-01). Số điện thoại được bảo vệ mặc định (BR-02).
University	Trường đại học nơi xảy ra sự việc mất hoặc nhặt được vật phẩm.	universityId, universityName, universityCode, address, status (APPROVED, PENDING)	Các trường/cơ sở mới do đại diện trường đề xuất cần được Quản trị viên xét duyệt trước khi hiển thị (FR-08).
ItemCategory	Danh mục phân loại vật phẩm (ví dụ: Thẻ sinh viên, Thiết bị điện tử, Ví/Bóp, Chìa khóa, Balo/Túi).	categoryId, categoryName, description	Do Quản trị viên quản lý tập trung; hỗ trợ phân loại cho bộ lọc tìm kiếm (FR-04).
Post	Thực thể trừu tượng đại diện cho một tin báo cáo vật phẩm thất lạc hoặc nhặt được trên khuôn viên trường.	postId, title, description, incidentDate, locationDetail, status (ACTIVE, PENDING_VERIFICATION, RESOLVED, CLOSED, REPORTED), createdAt	Mỗi bài đăng thuộc đúng một người tạo và gắn với một trường đại học; tự động chuyển sang hàng đợi nếu có ≥ 3 báo cáo vi phạm (BR-03).
LostPost	Chuyên biệt hóa của Post cho trường hợp sinh viên thông báo tài sản cá nhân bị thất lạc.	Kế thừa từ Post, bổ sung: rewardAmount (khoản hậu tạ tự nguyện), privateIdentifier (đặc điểm nhận dạng kín).	Chỉ chủ sở hữu bài đăng mới có thể cập nhật trạng thái bài đăng sang Resolved khi tìm lại được đồ.
FoundPost	Chuyên biệt hóa của Post cho trường hợp nhặt được vật phẩm và muốn tìm chủ sở hữu.	Kế thừa từ Post, bổ sung: storageLocation (nơi lưu giữ hiện tại: Bảo vệ/Tự giữ), securityQuestion (câu hỏi bảo mật).	Cung cấp câu hỏi bảo mật để người nhận (Claimant) chứng minh quyền sở hữu trước khi mở khóa thông tin liên hệ (BR-02).
ItemMedia	Hình ảnh thực tế của vật phẩm thất lạc hoặc khu vực xảy ra vụ việc.	mediaId, mediaUrl, fileType, uploadedAt	Mỗi bài đăng có thể đính kèm nhiều ảnh (1..*); ảnh thẻ sinh viên có thể che mờ thông tin riêng tư (FR-10).
Claim	Yêu cầu nhận lại vật phẩm do một sinh viên gửi tới một bài đăng Mất đồ hoặc Nhặt đồ.	claimId, claimMessage, proofImage, securityAnswer, status (PENDING, ACCEPTED, REJECTED, CANCELLED), submittedAt	Với FoundPost, Claim bắt buộc có câu trả lời cho câu hỏi xác minh, ảnh là tùy chọn. Với LostPost, Claim bắt buộc có mô tả nơi nhặt hoặc ảnh vật phẩm; chủ bài có quyền duyệt hoặc từ chối (FR-05, FR-06).

Report	Báo cáo phản ánh bài đăng có dấu hiệu vi phạm quy định (lừa đảo, thông tin sai sự thật, spam).	reportId, reason, detailMessage, status (PENDING, APPROVED, DISMISSED), reportedAt	Sinh viên chỉ được báo cáo 1 lần cho mỗi bài đăng; bài viết có ≥ 3 báo cáo tự động chuyển vào hàng đợi kiểm duyệt (BR-03).
Notification	Thông báo hệ thống gửi tới sinh viên nhằm cập nhật tiến trình nhận đồ hoặc tương tác tài khoản.	notificationId, title, messageBody, type (CLAIM_RECEIVED, CLAIM_DECIDED, POST_MODERATED), isRead, createdAt	Gửi tự động cho người dùng đích khi có sự kiện tương tác Claim hoặc thay đổi trạng thái bài đăng (FR-09).

4.1.2 Domain Relationships and Multiplicities

Các mối quan hệ cấu trúc và ngữ nghĩa giữa các thực thể trong mô hình lĩnh vực được tóm tắt trong bảng sau:

Thực thể nguồn	Thực thể đích	Tên quan hệ	Bội số	Ý nghĩa nghiệp vụ
University	Post	hosts / located_at	1 : 0..*	Một trường đại học có thể diễn ra nhiều bài đăng mất/nhặt đồ; mỗi bài đăng bắt buộc gắn với đúng 1 trường đại học.
ItemCategory	Post	classifies	1 : 0..*	Một danh mục chứa nhiều bài đăng; mỗi bài đăng được phân loại vào đúng 1 danh mục vật phẩm.
Post	ItemMedia	contains	1 : 1..*	Một bài đăng chứa ít nhất 1 ảnh minh họa thực tế; mỗi ảnh đính kèm thuộc về đúng 1 bài đăng.
User	Post	creates	1 : 0..*	Một người dùng có thể tạo nhiều bài đăng theo thời gian; mỗi bài đăng do đúng 1 người dùng sở hữu.
Post	LostPost / FoundPost	generalizes	1 : 1	Post là lớp cha trừu tượng khái niệm hóa các đặc tính chung của bài đăng mất đồ (LostPost) và nhặt đồ (FoundPost).

Post	Claim	receives	1 : 0..*	Một bài đăng Mất đồ hoặc Nhặt đồ có thể nhận nhiều yêu cầu nhận đồ từ nhiều sinh viên khác nhau; mỗi Claim gửi đến đúng 1 Post.
User	Claim	submits	1 : 0..*	Một sinh viên có thể nộp nhiều yêu cầu Claim cho các tài sản thất lạc khác nhau; mỗi Claim do đúng 1 sinh viên gửi.
Post	Report	flagged_on	1 : 0..*	Một bài đăng có thể bị khiếu nại/báo cáo bởi nhiều người dùng khác nhau; mỗi Report trỏ tới đúng 1 bài đăng.
User	Report	files	1 : 0..*	Một người dùng có thể gửi báo cáo vi phạm đối với các bài đăng nghi vấn; mỗi Report do đúng 1 người gửi.
User	Notification	receives	1 : 0..*	Một người dùng nhận nhiều thông báo trong quá trình tương tác; mỗi thông báo thuộc quyền sở hữu của 1 người dùng.

4.1.3 Business Rules Verification in Domain Model

Bốn quy tắc nghiệp vụ then chốt từ PA2 được ánh xạ và xác minh trực tiếp trên các thực thể khái niệm của Domain Model:

1. **Verified School Email (BR-01):** Thực thể `User` quản lý thuộc tính `universityEmail` (bắt buộc đuôi tên miền giáo dục hợp lệ) và trạng thái `status = ACTIVE` sau khi hoàn tất xác thực OTP/link. Các mối quan hệ tạo bài (`creates`) và gửi yêu cầu nhận đồ (`submits`) chỉ hợp lệ đối với `User` đã qua kiểm tra chính danh này.
2. **Protected Contact Details (BR-02):** Thuộc tính liên hệ riêng tư (`phoneNumber`, email cá nhân) được mô hình hóa gắn liền với `User`, nhưng độc lập với thực thể `Post`. Không có liên kết trực tiếp nào bộc lộ thông tin liên hệ của chủ bài đăng cho các sinh viên khác; thông tin này chỉ được chia sẻ theo ngữ cảnh cho một `User` đóng vai trò Claimant khi thực thể `Claim` chuyển sang trạng thái `ACCEPTED`.
3. **Claim Approval Workflow (BR-04 / FR-05, FR-06):** Quan hệ giữa `Post` và `Claim` cho phép cả

bài Mất đồ và bài Nhặt đồ nhận nhiều yêu cầu đối chất. Trạng thái của thực thể `Claim` tuân thủ vòng đời nghiêm ngặt: `PENDING` → `ACCEPTED` / `REJECTED`. Khi một `Claim` được chấp thuận bởi chủ bài đăng, một liên kết trạng thái đồng thời kích hoạt chuyển `Post.status` sang `RESOLVED`, khép lại bài toán tìm đồ.

4. **Moderation Threshold (BR-03 / FR-07):** Quan hệ kết tập `Post` (1) -- (0..*) `Report` ghi nhận các khiếu nại từ cộng đồng. Bất biến nghiệp vụ quy định: nếu $\sum \text{Report} \geq 3$, bài đăng vi phạm sẽ tự động kích hoạt chuyển trạng thái `Post.status = REPORTED`, tạm thời thu hồi quyền hiển thị công khai trên thanh tìm kiếm và đưa vào danh sách kiểm duyệt ưu tiên của Quản trị viên; bài chỉ hiển thị lại nếu Admin quyết định giữ.

4.2 Class Diagram

Thiết kế chi tiết kế thừa các thực thể ở Mục 4.1 và quy tắc đã thống nhất ở Mục 2: Claim nhận bởi cả LostPost và FoundPost; chủ bài quyết định; đủ ba người báo cáo khác nhau thì bài tự động bị ẩn. Các lớp được chia thành domain, application service và infrastructure. Express controller chuyển HTTP request thành lời gọi service; controller không truy cập repository trực tiếp. Các hình dưới đây là các phần của cùng một class diagram, dùng chung tên lớp và kiểu dữ liệu.

Ký hiệu UML: dấu $-$ là private, $+$ là public, # là protected; đường liền biểu diễn association, tam giác rỗng là kế thừa/hiện thực interface, đường đứt có mũi tên là dependency. Thuộc tính private được đọc qua getter; thay đổi trạng thái phải qua operation có kiểm tra điều kiện. `UUID`, `Instant`, `String?`, `List<T>` lần lượt là định danh, thời điểm UTC, chuỗi tùy chọn và danh sách. DTO là cấu trúc dữ liệu vào/ra, không phải entity được xuất trực tiếp thành JSON.

Quyết định chi tiết hóa từ mô hình lĩnh vực

Điểm thiết kế	Quyết định áp dụng cho 4.2--4.6
Vòng đời Post	Giữ ACTIVE, RESOLVED, REPORTED và CLOSED. Chủ bài chỉ sửa/xóa/tự hoàn tất khi ACTIVE; RESOLVED chỉ đọc theo UC-06 E2; REPORTED chờ Admin xử lý. CLOSED biểu diễn xóa mềm bởi chủ bài hoặc gỡ bởi Admin; `closeReason` phân biệt hai trường hợp. PENDING_VERIFICATION trong 4.1 là giá trị dự phòng, không có phép chuyển trong phạm vi hiện tại: tạo bài hợp lệ đi thẳng vào ACTIVE.
Giữ bài sau kiểm duyệt	`previousStatus` lưu ACTIVE hoặc RESOLVED trước khi chuyển REPORTED. Admin giữ bài thì khôi phục giá trị này; các báo cáo đang chờ chuyển DISMISSED. Chỉ báo cáo PENDING mới tính vào ngưỡng, mỗi người vẫn chỉ được báo cáo một bài một lần trong toàn bộ lịch sử.
User và xác thực	Tách `isVerified` khỏi `status = ACTIVE/BANNED`; tài khoản chưa xác minh vẫn tồn tại. Guest là ngữ cảnh truy cập, không tạo bản ghi User. Mật khẩu, link xác minh và token do Supabase Auth quản lý; application không lưu bản sao mật khẩu.

Điểm thiết kế	Quyết định áp dụng cho 4.2--4.6
Trường và cơ sở	Thêm Campus thuộc University; mỗi Post chọn một Campus, từ đó xác định University. Đây là phân rã của quan hệ University--Post ở 4.1, phục vụ UC-03, UC-05 và UC-13.
Ảnh bài đăng	Thiết kế này giữ mức tối thiểu một ảnh của 4.1, kết hợp giới hạn UC-05: 1--5 ảnh JPG/PNG, mỗi ảnh không quá 5 MB. Đây là quy tắc của bản thiết kế hiện tại, cần cập nhật đồng bộ nếu nhóm đổi sang ảnh tùy chọn.
Liên hệ hai bên	`ContactDetails` là value object riêng, không nằm trong PostDTO/ClaimDTO. Liên hệ chủ bài gắn với từng bài; liên hệ Claimant dùng email trường đã xác minh của User. Chủ bài được đọc liên hệ của chính bài mình; chỉ hai bên của Claim ACCEPTED được đọc liên hệ của đối phương.
Thuộc tính mở rộng	rewardAmount, privateIdentifier, storageLocation và studentId giữ là dữ liệu tùy chọn của 4.1; không tạo use case bắt buộc mới. privateIdentifier, storageLocation và liên hệ không được xuất ra DTO công khai.

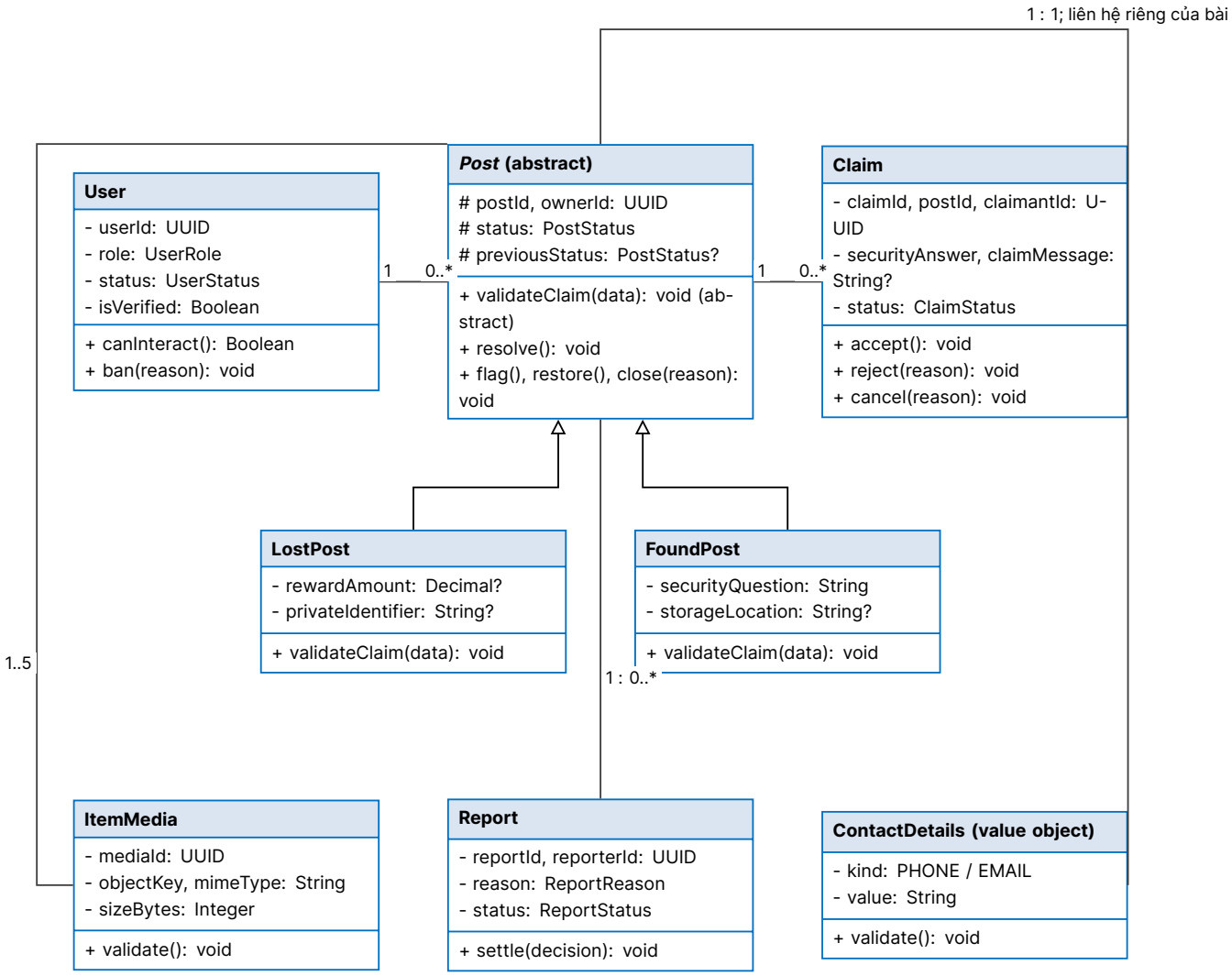


Figure 2: Các lớp domain cốt lõi và operation chuyển trạng thái

Hình 2 lược bỏ đường User--Claim và User--Report để tránh giao cắt: mỗi Claim/Report vẫn có đúng một người gửi; một User có thể gửi nhiều bản ghi như 4.1. Post và ảnh dùng association, không áp đặt xóa dây chuyền: xóa mềm bài vẫn giữ lịch sử Claim, Report và audit.

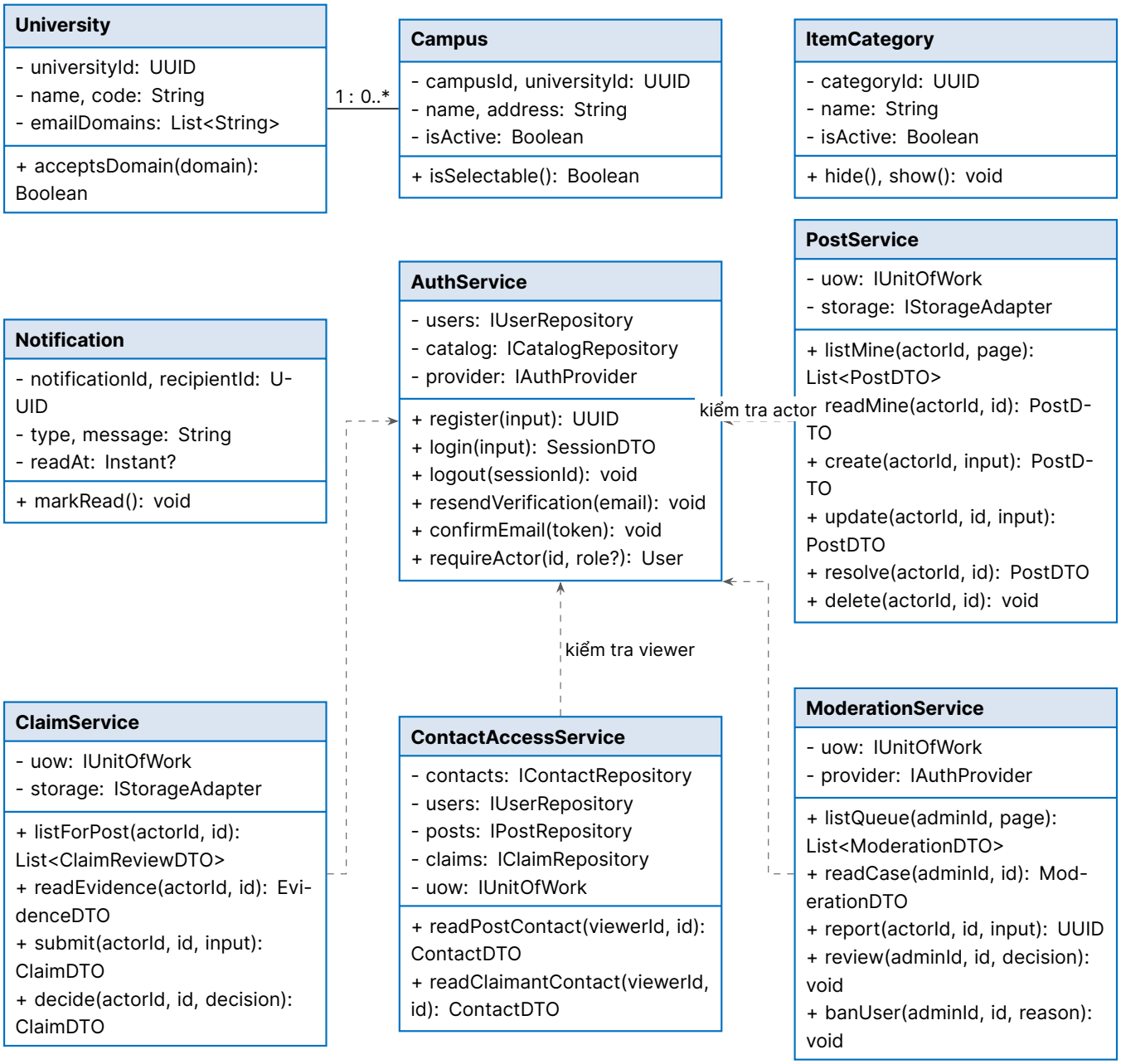


Figure 3: Các lớp hỗ trợ và application services; Campus và ItemCategory phân loại Post

PostService, ClaimService và ModerationService phụ thuộc NotificationService, AuthService và IUnitOfWork qua constructor injection. ContactAccessService chỉ đọc dữ liệu nhạy cảm qua IContactRepository; các service khác chỉ ghi liên hệ qua `IContactRepository.savePost()` và không đưa liên hệ vào DTO công khai. Notification thuộc đúng một User; mỗi Campus/ItemCategory liên kết

với nhiều Post, mỗi Post chọn đúng một Campus và ItemCategory.

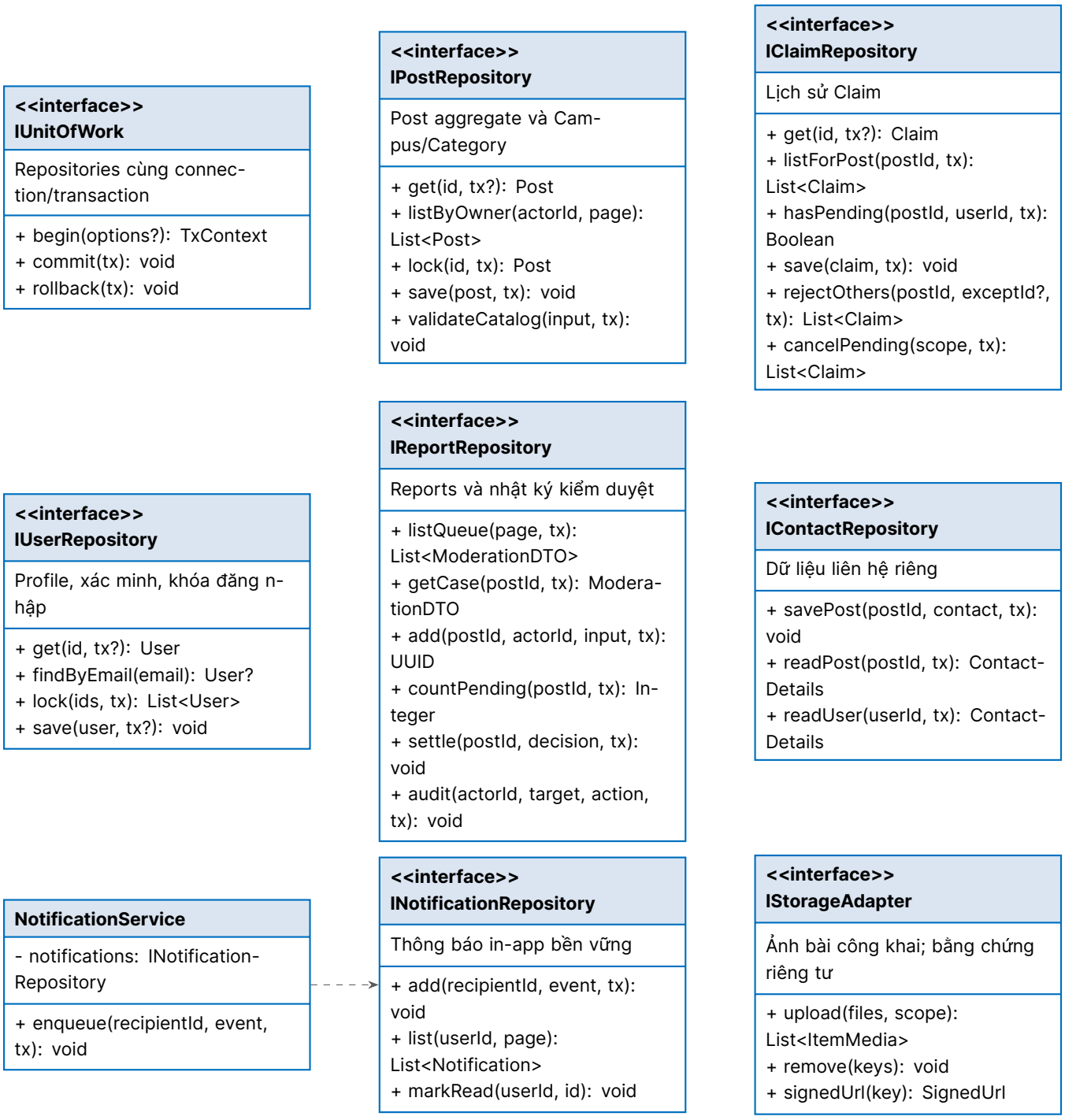


Figure 4: Các interface persistence, transaction, thông báo và storage

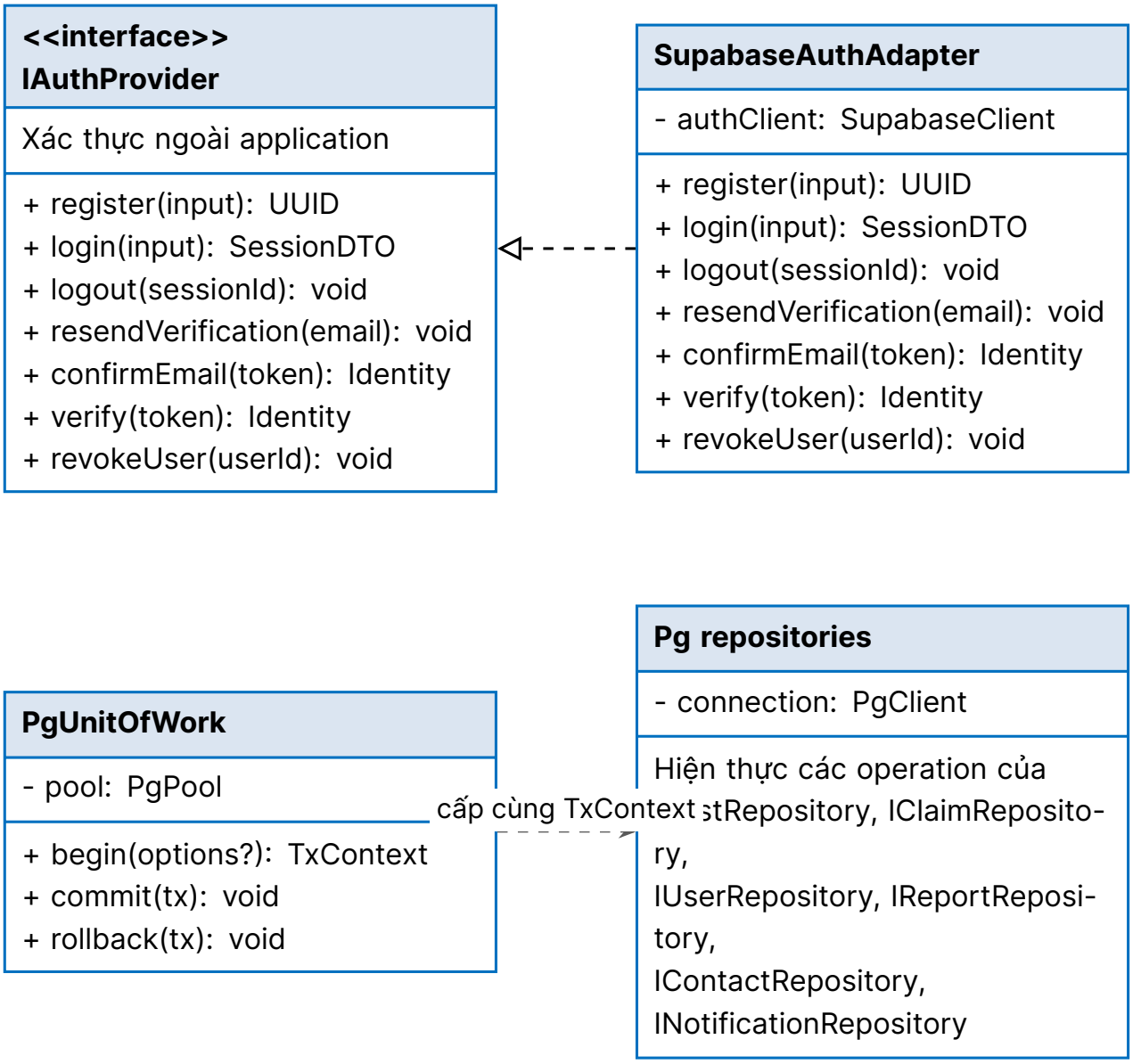


Figure 5: Hiện thực interface; SupabaseStorageAdapter hiện thực IStorageAdapter tương tự

PgUnitOfWork hiện thực IUnitOfWork; mỗi Pg repository hiện thực interface cùng tên nghiệp vụ (ví dụ PgClaimRepository hiện thực IClaimRepository). SupabaseStorageAdapter hiện thực toàn bộ operation của IStorageAdapter. Middleware gọi `IAuthProvider.verify(token)` để lấy identity, sau đó `AuthService.requireActor()` đọc lại profile hiện tại; role/status không được tin cậy chỉ từ payload phía client.

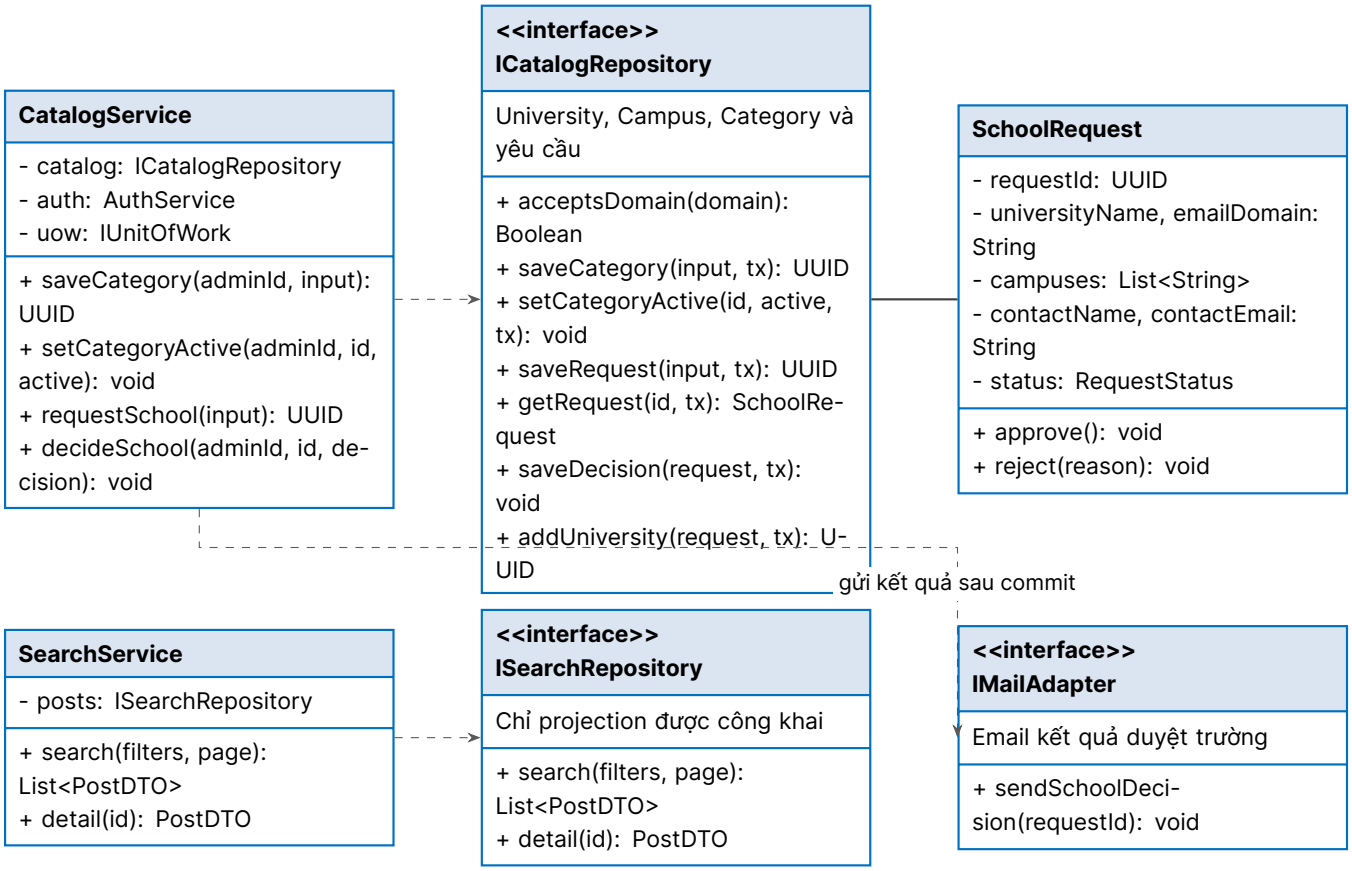


Figure 6: Các lớp bổ trợ cho tìm kiếm và quản lý danh mục/trường

Các lớp bổ trợ phủ FR-04/FR-08, không thuộc tám lớp đặc tả sâu. SearchService giới hạn 20 bài/trang, kết hợp bộ lọc bằng AND và dùng điều kiện công khai ở Mục 4.5. CatalogService kiểm tra ADMIN khi quản lý danh mục/duyệt trường; requestSchool công khai không cần User. CatalogService.decideSchool đọc getRequest, kiểm tra trạng thái PENDING, gọi SchoolRequest.approve/reject rồi lưu qua saveDecision; nhánh duyệt gọi addUniversity để tạo University, Campus và tên miền trong cùng transaction. Không chấp nhận tên miền trùng; email gửi sau commit qua IMailAdapter, lỗi thư không đảo kết quả và được phép gửi lại (UC-13 E4). Quyết định nghiệp vụ thuộc service/entity; repository chỉ truy vấn/lưu dữ liệu. Dữ liệu liên hệ đại diện trường chỉ thuộc luồng quản trị. AuthService đọc danh sách tên miền hiện tại qua ICatalogRepository.acceptsDomain, không dùng danh sách cố định trong SPA. PgCatalogRepository và PgSearchRepository hiện thực hai interface tương ứng.

Hợp đồng lỗi chung. Service kiểm tra actor và quyền sở hữu; dữ liệu sai trả ValidationError (HTTP 422), không có identity trả 401, không đủ quyền trả 403, đối tượng không tồn tại hoặc không được công khai trả 404, xung đột trạng thái/Claim trùng trả 409. Controller ánh xạ lỗi thành JSON và

không trả stack trace hay dữ liệu riêng tư. Các DTO chỉ chứa danh sách trường cho phép: PostDTO không chứa liên hệ/privateIdentifier/storageLocation; ClaimDTO không chứa liên hệ, câu trả lời/bằng chứng chỉ xuất cho chủ bài hoặc chính người gửi. Signed URL chỉ cấp sau kiểm tra quyền, không lưu URL có thời hạn trong entity.

4.3 Class Specifications

Tám lớp dưới đây là hợp đồng thiết kế cho các use case Must-have. Các getter/constructor được lược bỏ. Các service không có lớp cha; dependency là private, được truyền qua constructor và không đổi trong vòng đời service. Mọi định danh đầu vào phải được kiểm tra quyền, không coi biết UUID là được phép thao tác.

4.3.1 C1 -- User

Trách nhiệm: biểu diễn tài khoản và điều kiện tham gia nghiệp vụ (FR-01, FR-02, BR-01). Không có lớp cha. ACTIVE chưa có nghĩa là email đã xác minh: phải kiểm tra đồng thời `isVerified`.

Attribute	Type	Visibility	Constraint	Description
userId	UUID	private	Bắt buộc, duy nhất	Cùng identity với nhà cung cấp Auth.
fullName; universityEmail	String	private	Không rỗng; email chuẩn hóa và thuộc trường tham gia	Email định danh, không đưa vào DTO công khai.
role; status	UserRole; UserStatus	private	STUDENT/ADMIN; ACTIVE/BANNED	Guest không được lưu thành User.
isVerified	Boolean	private	Mặc định false; đồng bộ từ kết quả xác minh đáng tin cậy	Tách khỏi khóa tài khoản.
failedLoginCount; loginLockedUntil	Integer; Instant?	private	Đếm lần sai liên tiếp; lần thứ 5 khóa đăng nhập 15 phút	Hết hạn hoặc đăng nhập thành công thì đặt lại bộ đếm.
banReason; phoneNumber; studentId	String?	private	Lý do bắt buộc khi BANNED; số điện thoại nếu có phải hợp lệ	Không công khai dữ liệu liên hệ/định danh.

Operation	Parameters / return	Visibility	Contract
canInteract	Không tham số; Boolean	public	True khi ACTIVE và isVerified. Khóa đăng nhập tạm thời chỉ kiểm tra ở login, không tự đổi tài khoản sang BANNED.

Operation	Parameters / return	Visibility	Contract
ban	reason: String; void	public	Lý do không rỗng, User đích không phải ADMIN; chuyển BANNED. Điều kiện người gọi là Admin được kiểm tra ở service.

4.3.2 C2 -- Post (abstract)

Trách nhiệm: giữ bất biến của bài và chuyển trạng thái; LostPost/FoundPost kế thừa và override `validateClaim`. Service kiểm tra quyền trước khi gọi operation của entity (FR-03--FR-06).

Attribute	Type	Visibility	Constraint	Description
postId; ownerId; campusId; categoryId	UUID	protected	Bắt buộc; tham chiếu tồn tại	Không lưu thêm universityId khi đã suy ra từ Campus.
title; description; locationDetail	String; String?; String?	protected	Tiêu đề không rỗng	Nội dung tìm kiếm, không chứa liên hệ có cấu trúc.
incidentDate; createdAt	Instant	protected	incidentDate không ở tương lai tại thời điểm lưu	API kiểm tra lại khi tạo/sửa.
status; previousStatus	PostStatus; PostStatus?	protected	previousStatus chỉ ACTIVE/RESOLVED khi REPORTED	Không mất trạng thái giải quyết khi bị báo cáo.
closeReason; closedAt	String?; Instant?	protected	Bắt buộc khi CLOSED	OWNER_DELETED hoặc ADMIN_REMOVED, kèm lý do nghiệp vụ.
images	List<ItemMedia>	protected	1--5 ảnh; JPG/PNG, mỗi ảnh $\leq$ 5 MB	Chỉ lưu metadata/objectKey.
securityQuestion (FoundPost)	String	private	Không rỗng; không được đổi nếu có Claim PENDING	LostPost không có câu hỏi bắt buộc.

Operation	Parameters / return	Visibility	Contract
validateClaim	data: ClaimInput; void	public, abstract	FoundPost yêu cầu securityAnswer không rỗng; LostPost yêu cầu claimMessage hoặc proofImage. File nếu có phải hợp lệ. Chỉ ACTIVE nhận Claim mới.
resolve	Không tham số; void	public	ACTIVE $\rightarrow$ RESOLVED. Không mở liên hệ nếu không có Claim ACCEPTED.

Operation	Parameters / return	Visibility	Contract
flag	Không tham số; void	public	ACTIVE/RESOLVED → REPORTED; chụp previousStatus đúng một lần. Ngưỡng báo cáo được service tính.
restore	Không tham số; void	public	Chỉ từ REPORTED; phục hồi previousStatus rồi xóa giá trị dự phòng. Việc công khai còn phụ thuộc trạng thái chủ bài.
close	reason: CloseReason; void	public	ACTIVE → CLOSED khi chủ bài xóa; REPORTED → CLOSED khi Admin gỡ. Không có phép chủ bài xóa từ RESOLVED; lưu lý do và thời điểm, không hard-delete lịch sử.

4.3.3 C3 -- Claim

Trách nhiệm: ghi nhận yêu cầu kết nối và quyết định xử lý; không có lớp cha (FR-05, FR-06). Claim thuộc một Post bất kể loại bài; trạng thái đầu cuối không chuyển ngược về PENDING.

Attribute	Type	Visibility	Constraint	Description
claimId; postId; claimantId	UUID	private	Bắt buộc; claimant khác owner của Post	Mỗi cặp Post/User tối đa một PENDING; mỗi Post tối đa một ACCEPTED.
securityAnswer; claimMessage	String?	private	Điều kiện phụ thuộc Post.validateClaim	Câu trả lời chỉ đọc bởi người gửi và chủ bài.
proofImage	ItemMedia?	private	Tối đa một ảnh JPG/PNG ≤ 5 MB trong phạm vi này	ObjectKey ở bucket riêng tư, không phải URL công khai.
status; submittedAt	ClaimStatus; Instant	private	PENDING / ACCEPTED / REJECTED / CANCELLED	Mặc định PENDING.
decisionReason; decidedAt	String?; Instant?	private	Thời điểm có khi đã quyết định; lý do từ chối tùy chọn	Hủy do hệ thống có mã nguyên nhân.

Operation	Parameters / return	Visibility	Contract
accept	Không tham số; void	public	PENDING → ACCEPTED. Service phải kiểm tra chủ bài, bài ACTIVE và Claimant đang hoạt động trước khi gọi.

Operation	Parameters / return	Visibility	Contract
reject	reason: String?; void	public	PENDING → REJECTED; cũng dùng khi một Claim khác được chấp nhận hoặc chủ bài tự đánh dấu đã trả đồ.
cancel	reason: String; void	public	PENDING → CANCELLED khi bài bị xóa/gỡ hoặc người gửi bị khóa. Không thu hồi ACCEPTED bằng operation này.

4.3.4 C4 -- AuthService

Trách nhiệm: điều phối Supabase Auth, đồng bộ User và xác thực actor cho service (UC-01, UC-02). Không có lớp cha.

Attribute	Type	Visibility	Constraint	Description
users	IUserRepository	private	Constructor injection	Profile, bộ đếm đăng nhập và trạng thái hiện tại.
provider	IAuthProvider	private	Không lộ credential nhà cung cấp ra client	Xác minh link, session và mật khẩu ở adapter.
catalog	ICatalog Repository	private	Đọc danh sách tên miền đã được duyệt tại thời điểm đăng ký	Trường mới được duyệt có hiệu lực mà không sửa SPA.

Operation	Parameters / return	Visibility	Contract
register	input: RegisterInput; UUID	public	Kiểm tra tên miền trường đã tham gia, tên và mật khẩu tối thiểu 8 ký tự; tạo User chưa xác minh, gửi link hạn 24 giờ theo UC-01. Lỗi gửi thư giữ tài khoản để gửi lại.
login	input: LoginInput; SessionDTO	public	Kiểm tra khóa 15 phút trước khi gọi provider; từ chối chưa xác minh/BANNED. Năm lần sai liên tiếp kích hoạt khóa; không tiết lộ email hay mật khẩu sai.
logout	sessionId: UUID; void	public	sessionId lấy từ phiên đã xác thực của người gọi, không nhận ID phiên tùy ý từ body. Thu hồi phiên hiện tại; adapter duy trì trạng thái thu hồi đến khi token hết hạn, verify kiểm tra cả trạng thái này.

Operation	Parameters / return	Visibility	Contract
resendVerification	email: String; void	public	Theo UC-01 A1/E4/E6: gửi link mới, hạn 24 giờ, vô hiệu hóa link cũ; giới hạn tần suất. Lỗi gửi thư không đổi trạng thái tài khoản.
confirmEmail	token: String; void	public	Gọi provider.confirmEmail để kiểm tra link đúng loại, chưa dùng, còn hạn. Đồng bộ isVerified bằng identity đáng tin cậy; không dùng userId/email do client khai và không đổi BANNED thành ACTIVE.
requireActor	id: UUID, role?: UserRole; User	public	id lấy từ identity đã xác thực ở middleware; đọc User mới nhất, kiểm tra canInteract và role nếu có. Không dùng actorId do request body tự khai.

confirmEmail xử lý token xác minh một lần; IAuthProvider.verify chỉ xử lý access token của request API. Không trộn hai loại token. Khi đăng ký thành công ở provider nhưng ghi profile thất bại, đồng bộ lại theo cùng auth UUID bằng thao tác lưu idempotent; profile chưa tồn tại/đồng bộ chưa xong thì từ chối quyền nghiệp vụ. Đăng ký lại bằng email đã có vẫn xử lý theo UC-01 E2, không tạo hai User.

4.3.5 C5 -- PostService

Trách nhiệm: điều phối UC-05/UC-06, bảo toàn Post aggregate và liên hệ riêng. Không có lớp cha.

Attribute	Type	Visibility	Constraint	Description
auth; uow	AuthService; IUnitOfWork	private	Constructor injection	Kiểm tra actor; cung cấp repositories dùng chung transaction.
storage; notifications	IStorageAdapter; NotificationSer-vice	private	Không gọi dịch vụ mạng trong transaction CSDL	Ảnh và thông báo in-app.

Operation	Parameters / return	Visibility	Contract
listMine	actorId: UUID, page: PageInput; List<PostDTO>	public	Chỉ lấy bài có ownerId bằng actor đã xác thực, gồm trạng thái để quản lý lịch sử; mỗi trang tối đa 20 bài, mới nhất trước. Không dùng điều kiện công khai của SearchService.

Operation	Parameters / return	Visibility	Contract
readMine	actorId, id: UUID; PostDTO	public	Chỉ chủ bài; được đọc bài của mình để quản lý lịch sử kể cả RESOLVED/REPORTED/CLOSED. Quyền đọc riêng không cấp quyền sửa/xóa; không trả liên hệ trong PostDTO.
create	actorId: UUID, input: PostInput; PostDTO	public	Actor đã xác minh; Campus/Category được chọn; kiểm tra dữ liệu và ảnh. Chọn subtype theo loại bài; lưu Post ACTIVE, metadata và liên hệ trong một transaction. DTO không chứa liên hệ.
update	actorId, id: UUID, input: PostInput; PostDTO	public	Chỉ chủ bài, trạng thái ACTIVE; áp dụng validation như tạo. Giữ nguyên loại bài và ownerId; khóa câu hỏi xác minh khi có Claim PENDING.
resolve	actorId, id: UUID; PostDTO	public	Chỉ chủ bài ACTIVE. Gọi Post.resolve; từ chối Claim PENDING trong cùng transaction, thông báo người gửi. Áp dụng cho cả tự tìm lại đồ và tự trả đồ; không mở liên hệ.
delete	actorId, id: UUID; void	public	Chỉ chủ bài khi Post ACTIVE; xóa mềm bằng Post.close(OWNER_DELETED), hủy Claim PENDING và thông báo. RESOLVED chỉ đọc theo UC-06 E2; REPORTED chờ kiểm duyệt. Không xóa Claim đã kết thúc.

4.3.6 C6 -- ClaimService

Trách nhiệm: điều phối UC-07/UC-08 và giữ tính nguyên tử khi duyệt Claim. Không có lớp cha.

Attribute	Type	Visibility	Constraint	Description
auth; uow	AuthService; IUnitOfWork	private	Constructor injection	Quyền, repositories và transaction.
storage; notifications	IStorageAdapter; NotificationSer-vice	private	Bằng chứng riêng tư; notification cùng transaction	Không gửi liên hệ trong thông báo.

Operation	Parameters / return	Visibility	Contract
listForPost	actorId, id: UUID; List<ClaimReviewDTO>	public	Chỉ chủ Post được xem danh sách Claim, câu trả lời và metadata bằng chứng (UC-08 bước 1--3). Trả danh sách rỗng khi chưa có Claim; không cấp URL bằng chứng công khai hoặc liên hệ.
readEvidence	actorId, id: UUID; EvidenceDTO	public	id là claimId; chỉ chủ Post hoặc chính Claimant đang có tài khoản hợp lệ. Kiểm tra quan hệ trước khi gọi storage.signedUrl. Không có ảnh trả 404; có ảnh trả URL ngắn hạn và expiresAt, không lưu URL vào Claim.
submit	actorId, id: UUID, input: ClaimInput; ClaimDTO	public	Bài ACTIVE; actor khác chủ bài; hai tài khoản hoạt động. Kiểm tra bằng chứng theo subtype, không có PENDING trùng. Lưu PENDING và thông báo chủ bài nguyên tử.
decide	actorId, id: UUID, decision: ClaimDecision; ClaimDTO	public	Chỉ chủ bài, Claim PENDING, Post ACTIVE. ACCEPT: Claimant hợp lệ; chấp nhận một, Post.resolve, rejectOthers và thông báo tất cả trong một transaction. REJECT: chỉ từ chối Claim đích, bài vẫn ACTIVE.

ClaimDecision gồm action ACCEPT/REJECT và reason tùy chọn khi REJECT. Lặp lại hoặc hai request duyệt đồng thời không tạo quyết định mới: request mất quyền chuyển trạng thái nhận 409. Không gửi liên hệ trong ClaimDTO; sau commit, UI gọi ContactAccessService để đọc theo quyền hiện tại.

4.3.7 C7 -- ContactAccessService

Trách nhiệm: cổng đọc liên hệ được bảo vệ theo BR-02/NFR-02. Không có lớp cha. Kiểm tra cả actor, đối tượng và quan hệ; không có quyền đọc mặc định cho Admin.

Attribute	Type	Visibility	Constraint	Description
auth; contacts	AuthService; IContactRepository	private	Constructor injection	Xác thực viewer và truy cập dữ liệu riêng.
users; posts; claims	IUserRepository; IPostRepository; IClaimRepository	private	Đọc trạng thái hiện tại trong cùng snapshot	Không ra quyết định theo dữ liệu cache cũ.

Attribute	Type	Visibility	Constraint	Description
uow	IUnitOfWork	private	Transaction chỉ đọc, cùng snapshot cho kiểm tra và lấy liên hệ	Kết thúc transaction trước khi trả ContactDTO.

Operation	Parameters / return	Visibility	Contract
readPostContact	viewerId, id: UUID; ContactDTO	public	Chủ bài đọc liên hệ của chính bài; viewer khác chỉ được đọc nếu là Claimant ACCEPTED của đúng Post, hai tài khoản ACTIVE và bài RESOLVED. Còn lại 403/404.
readClaimantContact	viewerId, id: UUID; ContactDTO	public	id là claimId; viewer phải là chủ Post, Claim ACCEPTED, Post RESOLVED, hai tài khoản ACTIVE. Trả email trường đã xác minh của Claimant; không trả toàn bộ hồ sơ.

Hai endpoint đề xuất là `GET /posts/:id/contact` và `GET /claims/:id/contact`. Không thể mở liên hệ chỉ bằng việc chủ bài tự đánh dấu đã tìm/trả đồ, hoặc chỉ vì một Claim khác trên bài đã được chấp nhận. Response liên hệ đặt `Cache-Control: no-store`; log và notification không ghi giá trị liên hệ.

4.3.8 C8 -- ModerationService

Trách nhiệm: UC-09/UC-10, ngưỡng báo cáo và nhật ký Admin. Không có lớp cha.

Attribute	Type	Visibility	Constraint	Description
auth; uow	AuthService; IUnitOfWork	private	Constructor injection	Phân quyền, reports, posts, users và audit.
provider; notifications	IAuthProvider; NotificationSer-vice	private	Thu hồi phiên sau commit; thông báo trong transaction	BANNED có hiệu lực ở middleware ngay cả khi provider lỗi.

Operation	Parameters / return	Visibility	Contract
listQueue	adminId: UUID, page: PageInput; List<ModerationDTO>	public	Chỉ ADMIN; danh sách Post REPORTED kèm số reporter PENDING và lý do, số báo cáo giảm dần; danh sách rỗng là kết quả hợp lệ (UC-10 E3). Không lọc bằng quy tắc tìm kiếm công khai.

Operation	Parameters / return	Visibility	Contract
readCase	adminId, id: UUID; ModerationDTO	public	Chỉ ADMIN; đọc bài REPORTED và các báo cáo để đánh giá trước khi review. DTO không chứa liên hệ cá nhân hay bằng chứng riêng của Claim; không cấp quyền sửa bài trực tiếp.
report	actorId, id: UUID, input: ReportInput; UUID	public	Không báo cáo bài mình/CLOSED; một lần cho mỗi cặp Post/User. OTHER cần ghi chú. Thêm PENDING; đếm người báo cáo PENDING, đủ 3 gọi Post.flag và thông báo chủ bài. Tất cả cùng transaction.
review	adminId, id: UUID, decision: ReviewDecision; void	public	Chỉ ADMIN, bài REPORTED. KEEP: restore và settle DISMISSED. REMOVE: bắt buộc lý do, close ADMIN_REMOVED, settle APPROVED và cancel Claim PENDING. Ghi audit; thông báo các bên chịu tác động trong cùng transaction.
banUser	adminId, id: UUID, reason: String; void	public	Không khóa Admin; lý do bắt buộc. User.ban, hủy Claim PENDING do User gửi, ghi audit/thông báo và commit. Ẩn bài đang mở bằng điều kiện trên User, không đổi Post.status. Sau commit gọi revokeUser; lỗi được ghi để thử lại.

Hợp đồng repository bổ trợ. IPostRepository.listByOwner dùng cho listMine, còn get và kiểm tra ownerId dùng cho readMine. IClaimRepository.listForPost đọc Claim trong transaction chỉ đọc sau khi service đã kiểm tra quyền chủ bài. rejectOthers nhận exceptId tùy chọn: bỏ qua Claim được chấp nhận, hoặc null khi chủ bài tự hoàn tất. rejectOthers và cancelPending trả `List<Claim>` thực sự vừa đổi trạng thái; service lấy claimantId từ danh sách này để tạo đúng notification. cancelPending nhận ClaimScope gồm bộ lọc postId hoặc claimantId và mã lý do hủy. Không gửi thông báo lại cho Claim đã kết thúc trước đó. IReportRepository.listQueue/getCase chỉ cung cấp projection cho service quản trị, không mở endpoint đọc trực tiếp repository.

Snapshot khi đọc liên hệ. ContactAccessService bắt đầu transaction chỉ đọc với mức REPEATABLE_READ qua IUnitOfWork.begin(options). Các lệnh IUserRepository.get, IPostRepository.get, IClaimRepository.get/listForPost và IContactRepository.readPost/readUser đều nhận cùng TxContext cho việc kiểm tra quan hệ và lấy giá trị liên hệ; kết thúc transaction trước khi trả response. Các transaction ghi dùng READ_COMMITTED kết hợp khóa và kiểm tra lại trạng thái. Actor đã xác thực từ middleware vẫn phải được kiểm tra trạng thái hiện tại trong snapshot này.

Transaction và xử lý đồng thời. TxContext cung cấp cùng một PgClient cho mọi repository trong transaction; notification in-app được thêm qua NotificationService.enqueue trước commit. Thao tác gửi/duyệt Claim, sửa/đóng bài, report và review khóa cùng bản ghi Post trước khi kiểm tra trạng thái; việc kiểm tra số Claim PENDING cũng nằm sau khóa này. Các thao tác cần trạng thái User phối hợp khóa User liên quan theo UUID tăng dần trước khi khóa Post; banUser dùng cùng quy ước. Vì vậy khóa User và duyệt Claim không thể vượt qua nhau bằng kết quả kiểm tra cũ. Khóa ngoại và unique index ở section 5 sẽ bảo vệ bổ sung: một PENDING cho mỗi cặp Post/User, một ACCEPTED cho mỗi Post, một Report cho mỗi cặp Post/User.

Ranh giới tài nguyên ngoài CSDL. Upload ảnh diễn ra trước transaction; khi kiểm tra lại hoặc lưu thất bại, rollback rồi xóa các object mới đã upload. Storage adapter phải dọn cả phần upload thành công nếu một batch bị lỗi; object dọn thất bại được ghi nhận để thử lại. Không coi rollback PostgreSQL là rollback Storage/Auth. Khóa tài khoản được commit trước khi gọi provider thu hồi phiên; kiểm tra User tại mỗi request vẫn chặn được tài khoản ngay lập tức. Callback xác minh chỉ đồng bộ isVerified sau khi adapter đã xác thực bằng chứng của provider.

4.4 Sequence Diagrams

Năm use case UC-05, UC-07, UC-08, UC-09 và UC-10 được chọn vì bao phủ tạo bài, kết nối nhận đồ và kiểm duyệt. Lifeline UI/HTTP boundary gom thao tác người dùng và controller mỏng; các lifeline còn lại là lớp/interface đã khai báo ở Mục 4.2. Đường liền là lời gọi operation, đường đứt là kết quả trả về. Vùng alt/else là các nhánh loại trừ nhau; opt chỉ chạy khi điều kiện đúng. Các ghi chú begin/commit/rollback tương ứng operation của IUnitOfWork, không phải lời gọi đến một database ngoài kiến trúc.

Middleware xác thực phiên trước khi service nhận actorId. Guard quyền, kiểm tra trạng thái mới nhất và quy ước khóa ở Mục 4.3 áp dụng cho mọi hình. NotificationService.enqueue gọi INotificationRepository.add trên cùng TxContext; chưa commit thì notification chưa được hiển thị. Các operation repository rejectOthers, cancelPending và settle áp dụng phép chuyển domain tương ứng cho từng bản ghi, chỉ đổi những bản ghi còn PENDING. Hai operation đầu trả danh sách Claim vừa thay đổi để service xác định người cần thông báo. Mỗi use case có đúng một sequence diagram; tổng cộng năm hình, trong đó UC-08 và UC-10 bao gồm cả bước đọc dữ liệu trước khi quyết định.

4.4.1 UC-05 -- Tạo bài đăng Mất/Nhặt đồ

Tiền điều kiện: tài khoản đã xác minh, không BANNED. Service kiểm tra PostInput gồm loại bài, tiêu đề, Campus, Category, thời gian, ảnh và liên hệ; FoundPost cần câu hỏi xác minh. Đây là hậu kiểm: bài hợp lệ được lưu ACTIVE. Dữ liệu không hợp lệ trả 422 trước khi lưu; storage lỗi không tạo bài; lỗi CSDL sau upload kích hoạt dọn ảnh. Kết quả công khai không chứa liên hệ.

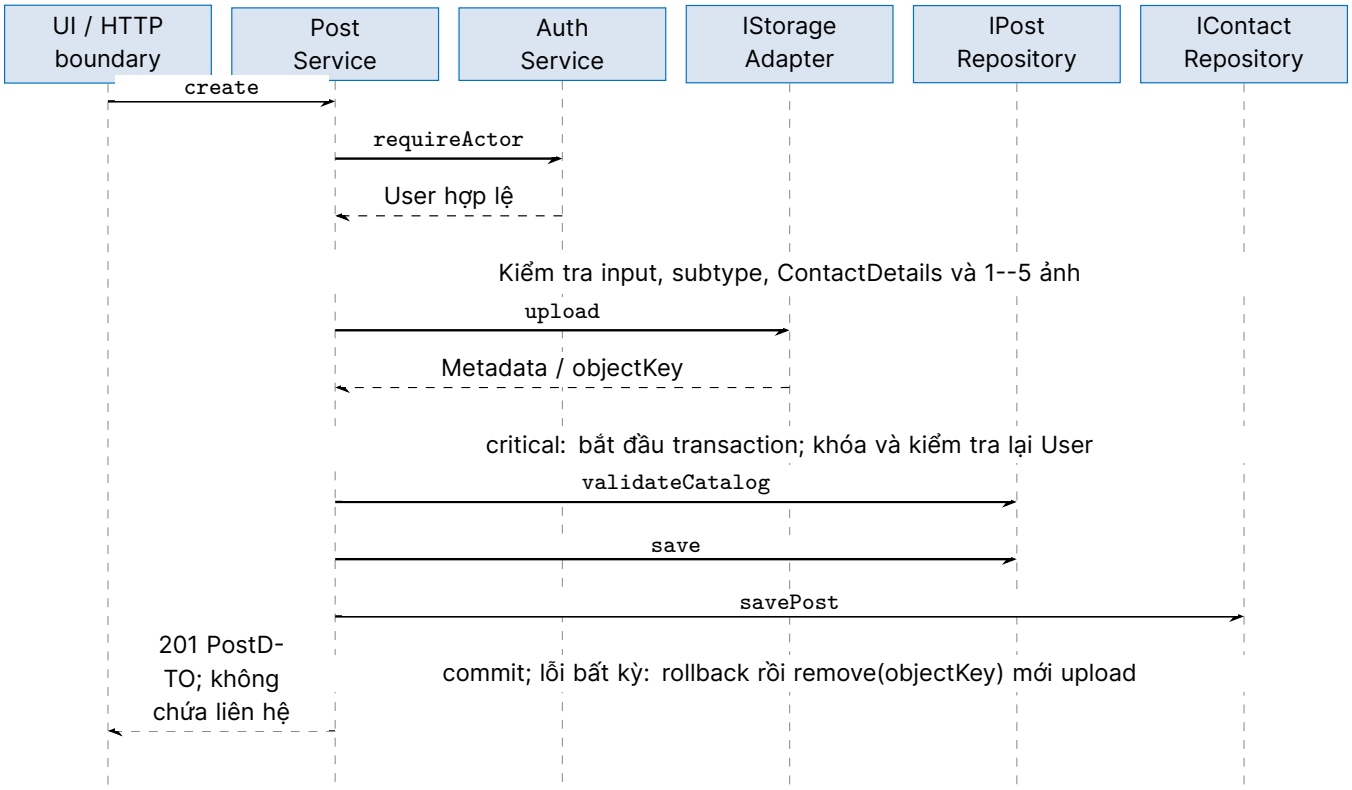


Figure 7: UC-05 -- Tạo bài và liên hệ riêng, không có bước duyệt trước

4.4.2 UC-07 -- Gửi Claim

LostPost.validateClaim yêu cầu mô tả nơi nhặt hoặc ảnh; FoundPost.validateClaim yêu cầu câu trả lời, có thể thêm ảnh. Dù UI đã kiểm tra trước upload, service vẫn kiểm tra lại trạng thái Post/User sau khóa để tránh bài đã đóng hoặc tài khoản vừa bị khóa. Bằng chứng được upload vào scope riêng tư; thiếu ảnh thì bỏ qua upload. Hủy form không gọi submit và không tạo bản ghi. Claim trùng, tự Claim hoặc bài không còn nhận yêu cầu đều không sinh notification.

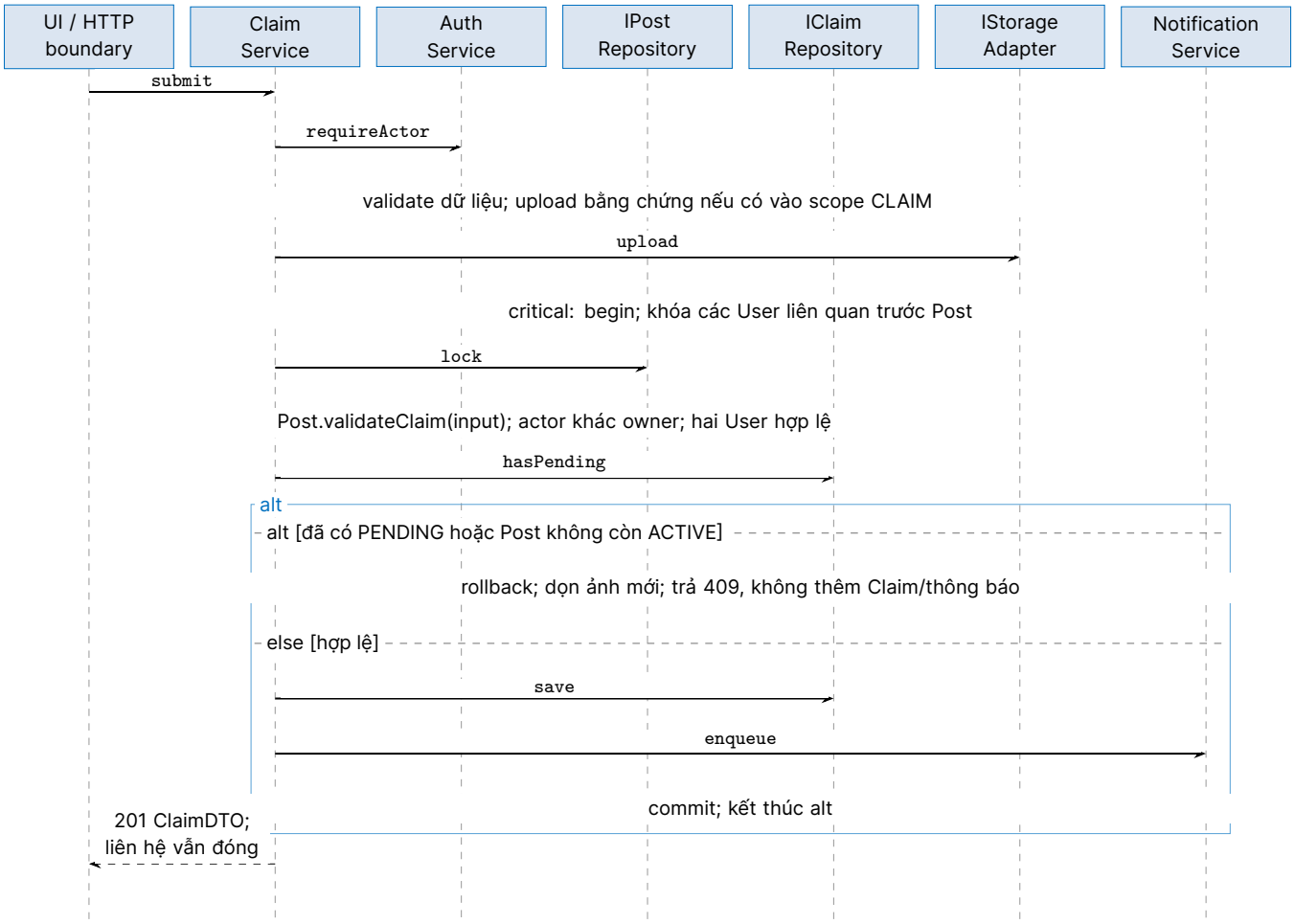


Figure 8: UC-07 -- Gửi Claim cho hai loại bài; chặn Claim đang chờ trùng

4.4.3 UC-08 -- Chấp nhận hoặc từ chối Claim

Chủ bài mở danh sách qua ClaimService.listForPost; service xác thực actor, đọc Post và kiểm tra ownerId trước khi trả câu trả lời. Danh sách rỗng hiển thị trạng thái chưa có yêu cầu (UC-08 E1). Khi bấm ảnh, readEvidence kiểm tra lại actor và quan hệ với Claim trước khi cấp signed URL ngắn hạn. Không dùng SearchService.detail công khai để đọc bằng chứng, cũng không cấp quyền chỉ vì người gọi biết claimId.

Hộp thoại xác nhận ở UI phải hoàn tất trước khi gửi quyết định ACCEPT. Nhánh ACCEPT chỉ chạy nếu Claimant vẫn hợp lệ. Một Claim được nhận, Post chuyển RESOLVED và các Claim PENDING khác chuyển REJECTED trong cùng transaction. Các Claim đã kết thúc từ trước được giữ nguyên lịch sử. Chỉ sau commit, hai người dùng mới có thể đọc liên hệ qua các request độc lập; service đọc

lại quyền nên tài khoản bị khóa hoặc bài bị gỡ sau đó không tiếp tục được cấp liên hệ đối phương.

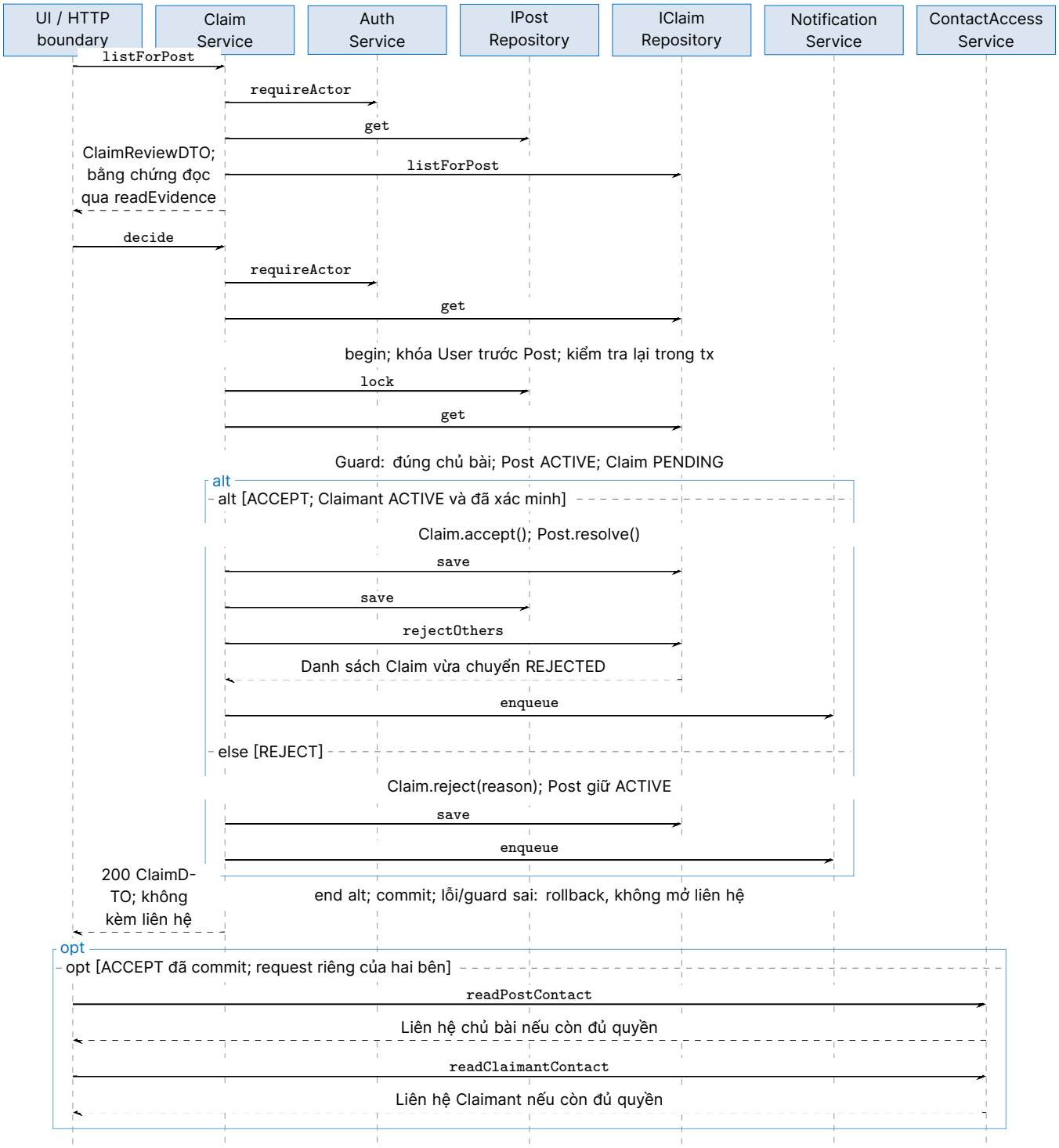


Figure 9: UC-08 -- Xem, quyết định Claim và đọc liên hệ sau commit

Hai lần duyệt đồng thời: request đầu giữ khóa Post và commit; request sau đọc lại RESOLVED/Claim đã quyết định rồi trả 409. Unique index cho một ACCEPTED/Post là lớp bảo vệ cuối cùng; không chỉ dựa vào việc UI ẩn nút. Nếu bất kỳ update/notification thất bại, rollback toàn bộ và liên hệ vẫn không đủ điều kiện mở.

4.4.4 UC-09 -- Báo cáo vi phạm

Mỗi reporter chỉ tạo một Report cho một Post trong toàn bộ lịch sử. OTHER cần ghi chú. Hình minh họa ngưỡng ba theo quyết định section 2/4.1, bổ sung chi tiết tự động ẩn so với mô tả hàng đợi trong UC-09 PA2. Báo cáo đã settle không còn tính vào countPending; việc Admin giữ bài không xóa lịch sử reporter. Post đã REPORTED không ghi đè previousStatus hoặc phát lại cảnh báo mỗi khi có người báo cáo thêm.

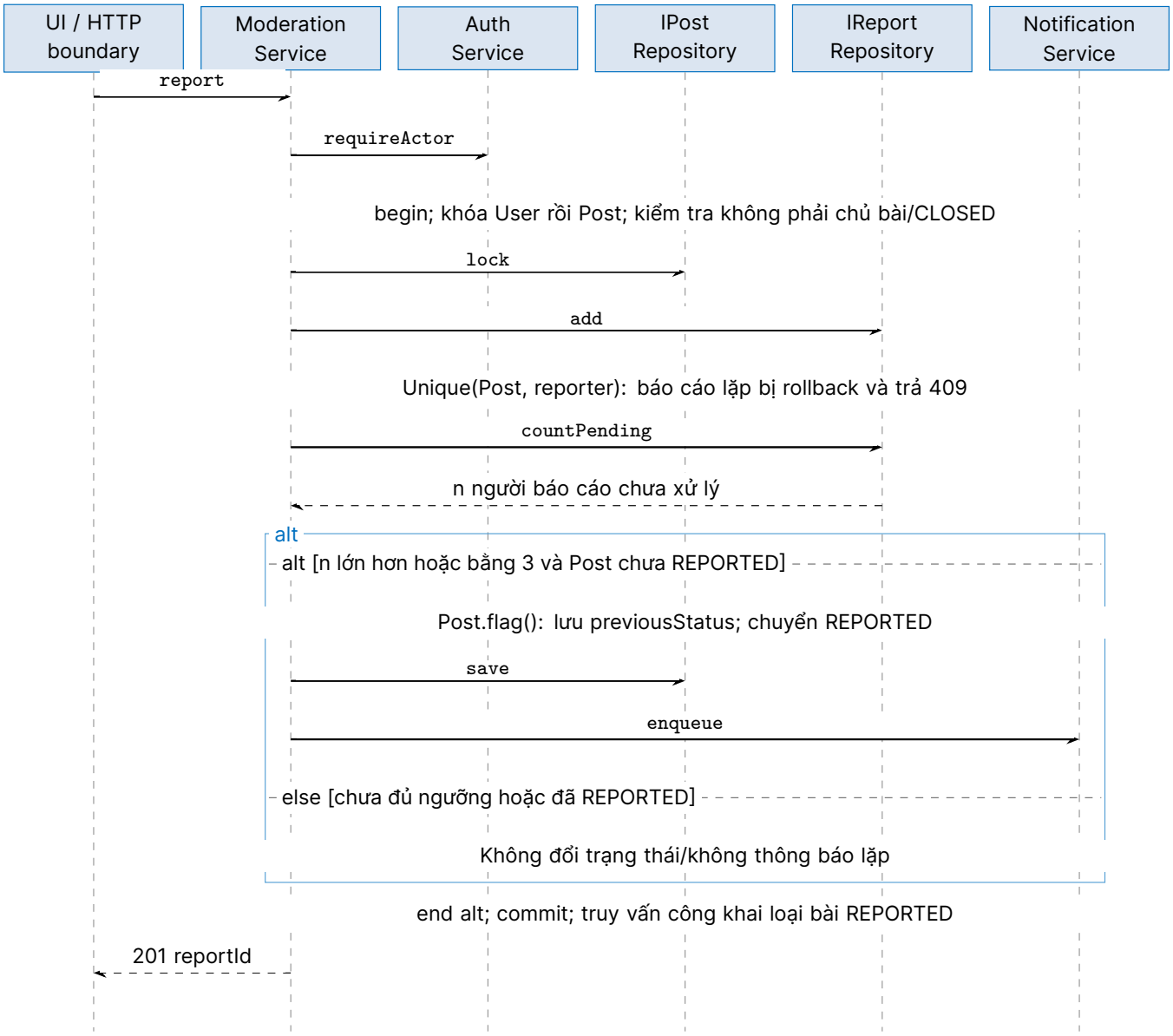


Figure 10: UC-09 -- Đếm người báo cáo khác nhau và tự động ẩn tại ngưỡng ba

4.4.5 UC-10 -- Kiểm duyệt và khóa tài khoản

Admin mở hàng đợi bằng listQueue, rồi đọc bài bị ẩn và các báo cáo qua readCase. Cả hai operation đều kiểm tra role ADMIN; không dùng SearchService.detail vốn loại bài REPORTED. Nếu hàng đợi rỗng, UI hiển thị thông báo UC-10 E3. ModerationDTO chỉ chứa dữ liệu cần kiểm duyệt, không kèm liên hệ riêng hoặc bằng chứng Claim.

Nhánh KEEP khôi phục trạng thái trước khi bị báo cáo; nếu chủ bài BANNED thì bài vẫn không công khai theo điều kiện hiển thị. Nhánh REMOVE yêu cầu lý do, đóng bài, hủy Claim PENDING và lưu audit. Hai Admin xử lý cùng một bài được tuần tự hóa bằng khóa Post; người xử lý sau nhận 409 nếu bài không còn REPORTED.

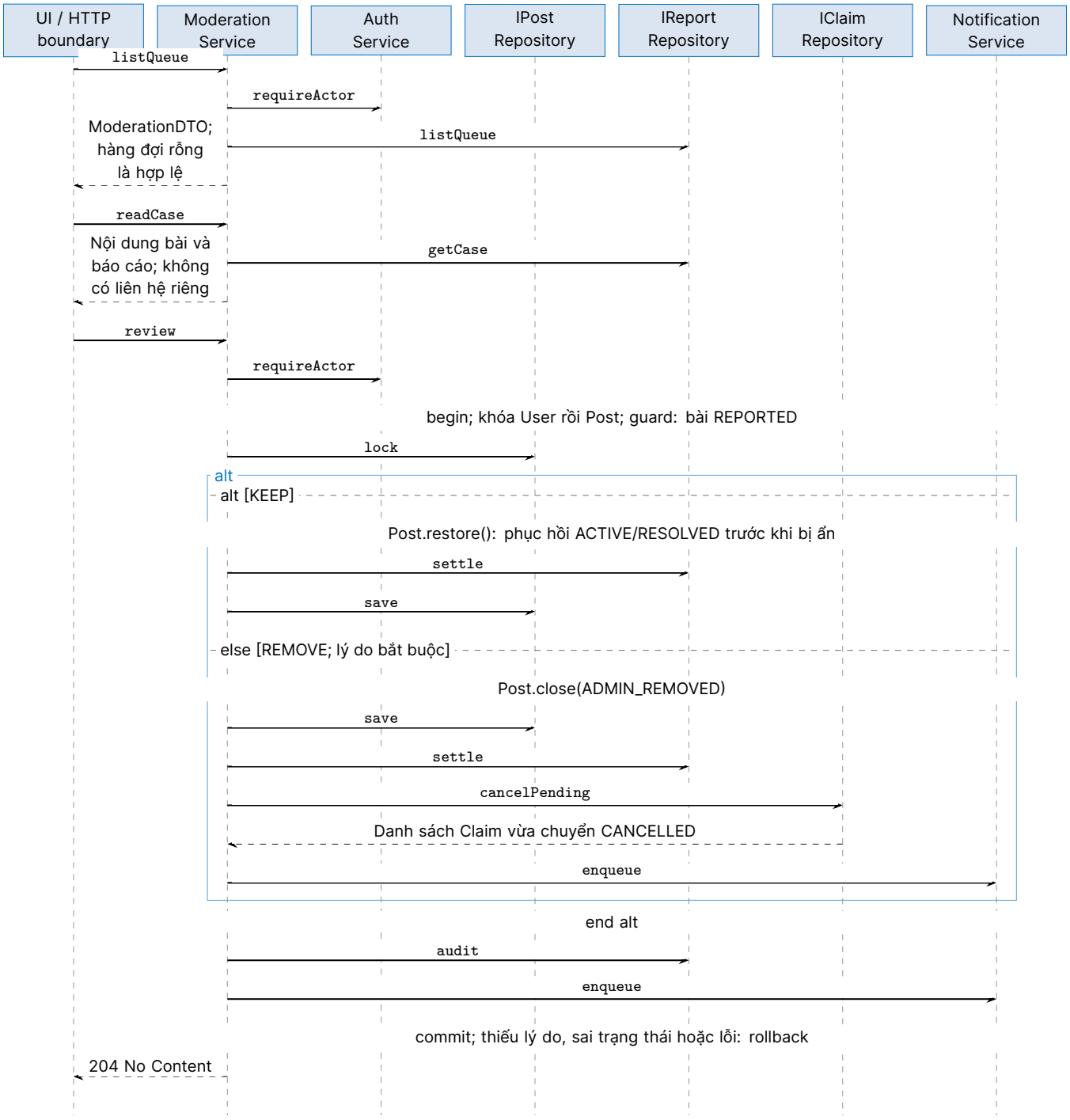


Figure 11: UC-10 -- Đọc hàng đợi, giữ/gỡ bài và lưu audit nguyên tử

Luồng A2 khóa tài khoản dùng operation `ModerationService.banUser(adminId, userId, reason)`:

AuthService.requireActor kiểm tra ADMIN; IUserRepository.lock/get lấy User đích; từ chối nếu đích là ADMIN hoặc thiếu lý do; User.ban và IUserRepository.save lưu BANNED; IClaimRepository.cancelPending hủy Claim do tài khoản gửi; IReportRepository.audit và NotificationService.enqueue lưu nhật ký/thông báo, rồi commit. Sau đó IAuthProvider.revokeUser thu hồi các phiên ở provider. Nếu provider lỗi, giữ BANNED, ghi nhận lỗi để thử lại; middleware đọc trạng thái tài khoản vẫn chặn mọi request mới. Không thay Post.status chỉ để ẩn bài của tài khoản bị khóa.

4.5 State Machine Diagrams

Máy trạng thái dùng các operation domain ở Mục 4.2. Nhãn chuyển trạng thái có dạng *sự kiện [điều kiện] / tác động*; service thực thi kiểm tra quyền và transaction. Enum được encapsulate trong entity, không có setter trạng thái tùy ý. Đây là mô hình hành vi; chưa yêu cầu tạo một lớp cho từng trạng thái.

4.5.1 Vòng đời Post

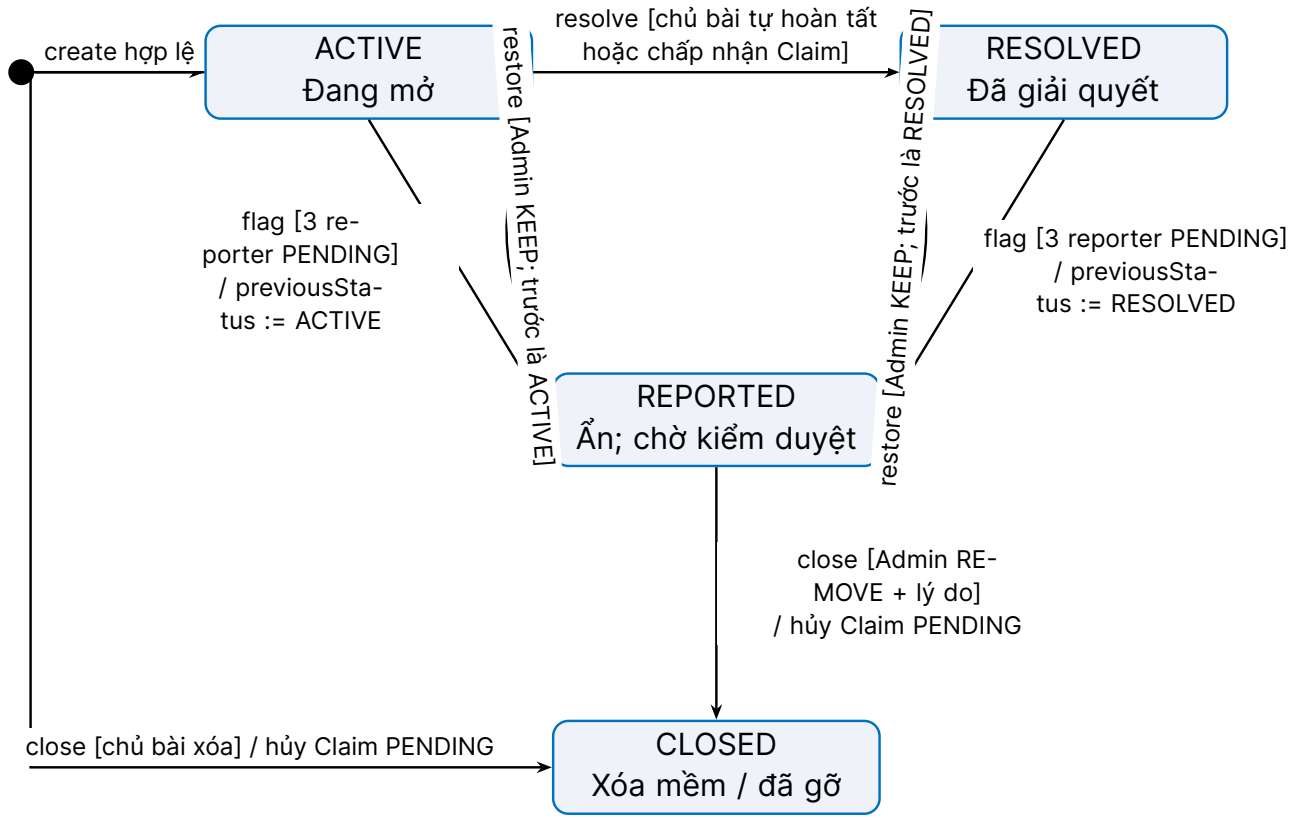


Figure 12: Post: khôi phục đúng trạng thái trước khi vào hàng đợi

PENDING_VERIFICATION không phải trạng thái khởi tạo trong phạm vi hậu kiểm hiện tại; giá trị này của 4.1 được dự phòng, không persist bằng luồng tạo bài. CLOSED là trạng thái kết thúc; không có operation mở lại. Chủ bài chỉ sửa/xóa/tự hoàn tất khi ACTIVE. RESOLVED chỉ được xem theo UC-06 E2; REPORTED chờ Admin giữ/gỡ, chủ bài không sửa/xóa để vượt qua hàng đợi. Một bài RESOLVED vẫn có thể bị báo cáo, chuyển REPORTED và sau đó bị Admin gỡ. Phân biệt nhãn ``Đã xóa'' và ``Đã gỡ'' bằng closeReason.

Hiển thị công khai: chỉ ACTIVE/RESOLVED, chủ bài không BANNED; REPORTED/CLOSED bị loại khỏi danh sách và chi tiết công khai. Điều kiện gửi/duyệt Claim chặt hơn: chỉ ACTIVE và các tài khoản liên quan hợp lệ. Khi ACTIVE được chủ bài tự hoàn tất, Claim PENDING chuyển REJECTED; không có ACCEPTED nên không mở liên hệ. Việc giữ bài đã RESOLVED khôi phục trạng thái và các quyền đọc liên hệ chỉ nếu Claim ACCEPTED cùng tài khoản vẫn hợp lệ.

4.5.2 Vòng đời Claim

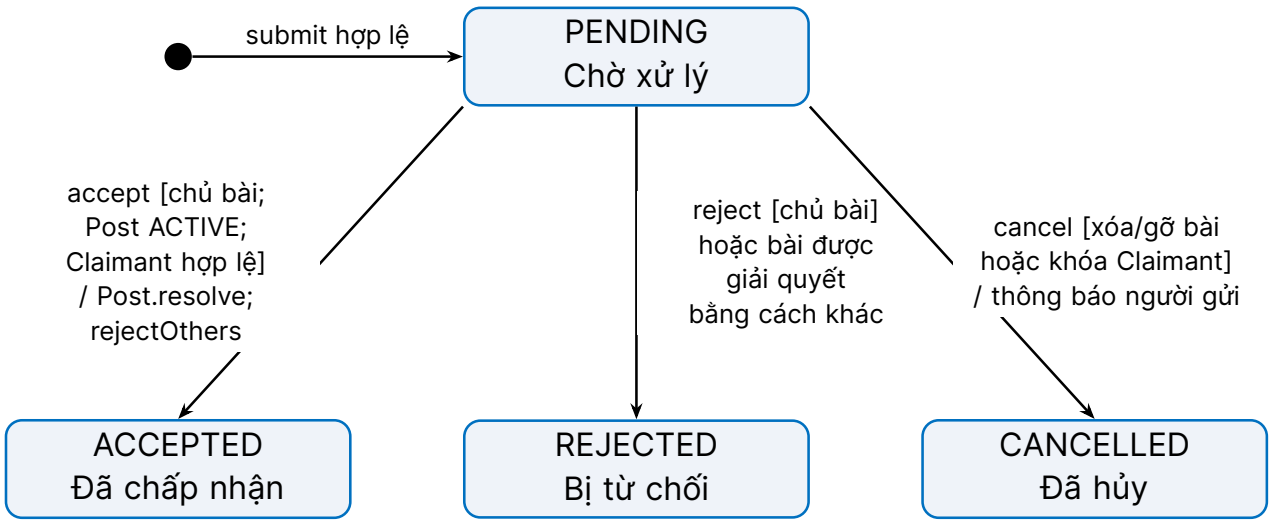


Figure 13: Claim: ba kết quả cuối cùng, không quay về PENDING

Chỉ Claim PENDING mới được đổi trạng thái. Giữ nguyên ACCEPTED/REJECTED/CANCELLED khi xử lý hàng loạt; không xóa lịch sử để gửi lại. Người gửi có thể tạo Claim mới sau REJECTED nếu bài vẫn ACTIVE và chưa có PENDING; không thể tạo thêm sau ACCEPTED vì bài đã RESOLVED. Nút ``Hủy'' trong form UC-07 đóng form trước submit, không phải operation hủy một Claim đã lưu. Không bổ sung use case rút Claim sau submit trong phạm vi này.

4.5.3 Vòng đời User và khóa đăng nhập

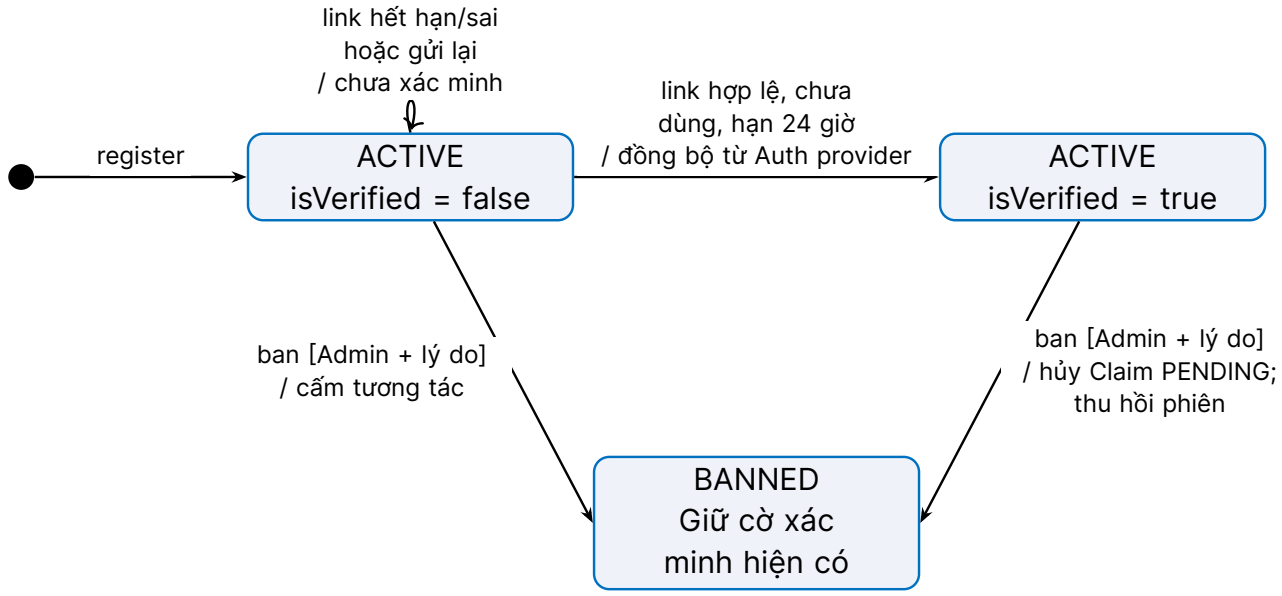


Figure 14: User: trạng thái tài khoản độc lập với xác minh email

Khóa đăng nhập 15 phút là bộ đếm và thời điểm `loginLockedUntil`, không phải BANNED. Sau năm lần sai liên tiếp, mọi login bị từ chối đến hạn; sau hạn đặt lại bộ đếm, đăng nhập thành công cũng đặt lại. Trạng thái xác minh không bị mất. BANNED không có phép mở khóa trong UC-10 hiện tại; không tự thêm operation unban. Role ADMIN do quản trị hệ thống cấp, không do form đăng ký tự chọn.

Invariant	Nơi thực thi	Tình huống kiểm tra ở PA4/PA5
Một ACCEPTED trên một Post	ClaimService.decide + khóa Post + unique index	Hai request chấp nhận đồng thời: chỉ một thành công.
Một PENDING cho mỗi cặp Post/User	ClaimService.submit + khóa Post + unique index có điều kiện	Gửi trùng bị 409; lịch sử REJECTED không cấm gửi lại khi bài còn ACTIVE.
Liên hệ đối phương cần Claim ACCEPTED	ContactAccessService và DTO theo danh sách trường	Khách, sinh viên khác, Claim PENDING/REJECTED không đọc được bằng API.
Ngưỡng ba người khác nhau	Report unique theo Post/User; countPending trong transaction	Người báo cáo thứ ba ẩn bài; giữ bài xong không bị báo cáo cũ tự ẩn lại.

Invariant	Nơi thực thi	Tình huống kiểm tra ở PA4/PA5
Chuyển trạng thái nguyên tử	IUnitOfWork; notification chung transaction	Gây lỗi giữa cập nhật Claim/Post: rollback, không mở liên hệ/thông báo sai.
Khóa tài khoản có hiệu lực ngay	User BANNED được kiểm tra trên mỗi request	Token còn hạn vẫn bị từ chối; không duyệt Claim của User vừa bị khóa.

4.6 Design Principles and Patterns

Các nguyên tắc và pattern dưới đây gắn với lớp và operation đã thể hiện ở 4.2--4.5. Mục tiêu là giữ quy tắc tập trung, kiểm thử được với thời hạn coding ba tuần; không triển khai thêm microservice hoặc hệ thống message broker.

Principle / pattern	Where it is applied	Why
Single Responsibility (SRP)	Post/Claim giữ bất biến; PostService/ClaimService điều phối; repository lưu dữ liệu; controller ánh xạ HTTP	Đổi giao thức API không làm đổi quy tắc duyệt Claim; thay truy vấn SQL không làm controller chứa nghiệp vụ.
Encapsulation	Thuộc tính trạng thái private/protected; Post.resolve/flag/restore/close và Claim.accept/reject/cancel	Chặn setter tùy ý đưa Claim đã từ chối thành ACCEPTED hoặc phục hồi bài CLOSED. Guard và hậu điều kiện có một nơi định nghĩa.
Polymorphism; Open/Closed	Post.validateClaim được LostPost/FoundPost override	Quy tắc bằng chứng khác nhau nằm trong subtype tương ứng. ClaimService.submit dùng Post chung, tránh chỉ nhận FoundPost hoặc rải kiểm tra loại bài khắp controller.
Service Layer	AuthService, PostService, ClaimService, ContactAccessService, ModerationService	Đặt phân quyền, transaction và phối hợp entity ở ranh giới use case; mọi đường gọi HTTP dùng cùng hợp đồng nghiệp vụ.
Repository; Dependency Inversion	Service phụ thuộc IPostRepository, IClaimRepository và các interface khác; Pg repositories hiện thực	Domain không phụ thuộc pg/SQL. Có thể thay bằng fake repository để kiểm thử điều kiện và lỗi, dùng PostgreSQL thật để kiểm thử transaction/unique.
Constructor Dependency Injection	Truyền AuthService, IUnitOfWork, IStorageAdapter và NotificationService khi khởi tạo service	Không new SupabaseClient trong mỗi use case; dễ thay adapter bằng fake để tạo lỗi upload, thu hồi phiên hoặc rollback trong kiểm thử.

Principle / pattern	Where it is applied	Why
Adapter	SupabaseAuthAdapter và SupabaseStorageAdapter	Đóng gói token/link/Storage SDK, tách credential hạ tầng khỏi domain. Auth và Storage không bị giả định tham gia transaction PostgreSQL.
Unit of Work	IUnitOfWork/PgUnitOfWork và TxContext; ClaimService.decide, ModerationService.review	Các repository và thông báo cùng dùng một transaction. Không có trường hợp Claim ACCEPTED nhưng Post chưa RESOLVED hoặc notification đã báo khi rollback.
State machine với transition guard	Enum PostStatus, ClaimStatus, UserStatus và các operation domain ở Mục 4.5	Hiện thực ý định State Pattern nêu ở section 2 bằng enum và operation có guard cho quy mô hiện tại. Chưa tạo một lớp concrete state cho mỗi giá trị vì chưa cần thêm hành vi phức tạp.
Least privilege; DTO theo allowlist	ContactAccessService đọc liên hệ; PostDTO/ClaimDTO không chứa liên hệ; bằng chứng chỉ có signed URL sau kiểm tra quyền	Không phụ thuộc vào việc giao diện che một ô dữ liệu; người gọi API trực tiếp cũng không được đọc thông tin chưa được cấp quyền (BR-02, NFR-02).
DRY; validation nhiều tầng	DTO kiểm tra hình thức; entity kiểm tra quy tắc theo loại Post; database giữ FK/unique/check	UI phản hồi sớm, API không tin dữ liệu client, CSDL chặn tranh chấp đồng thời. Mỗi tầng có trách nhiệm khác nhau, không chép một bộ quy tắc nghiệp vụ vào mọi controller.

Các lựa chọn này cần được phản ánh trong section 5 khi thiết kế bảng, index và transaction; các trường hợp kiểm tra ở Mục 4.5 là đầu vào kế hoạch PA4, chưa phải kết quả kiểm thử ứng dụng. Các quyết định chi tiết hóa ở đầu 4.2 phải được đưa vào truy vết/thay đổi yêu cầu của PA3 khi nhóm tích hợp báo cáo, đặc biệt quy tắc tối thiểu ảnh và cách biểu diễn CLOSED.

Đối chiếu thiết kế và bằng chứng

Class diagram tại Hình 2--6 được liên kết với tám đặc tả ở Mục 4.3. Năm sequence tại Hình 7--11 dùng các operation trong những lớp/interface này. Các phép chuyển Post/Claim tại Hình 12 và Hình 13 là hậu điều kiện của các luồng tạo, duyệt và kiểm duyệt tương ứng.

Trạng thái review components/ERD: chưa hoàn tất. Ở bản tài liệu hiện tại, section 3 chưa có Component Diagram và section 5 chưa có ERD/schema cụ thể. Vì vậy chưa thể xác nhận ranh giới component, FK, index và quyền đọc dữ liệu đã khớp với OOD. Bảng truy vết nguồn và danh sách cần đối chiếu được lưu tại docs/design/pa3-ood-traceability-review.md; đây là bằng chứng kiểm tra và đầu mục bàn giao, không thay thế review với hai thiết kế còn thiếu.

5 Data Design

5.1 Data Diagram

[ERD / database diagram of the stored data and their relationships (MySQL Workbench, dbdiagram.io, ...).]

5.2 Data Specification

[For a DBMS: describe each table. For JSON/XML/file storage: describe the structure, data types, constraints and give a sample file.]

Table	Column	Data type	Constraints (PK, FK, NOT NULL, UNIQUE, CHECK)	Description

5.3 Normalisation and Indexes

[State the normal form reached (at least 3NF, or justify denormalisation) and the indexes created for frequent queries.]

6 User Interface and User Experience Design

6.1 Screen List and Screen Flow

[Draw the screen flow diagram (navigation between screens).]

#	Screen	Description	Related use case

6.2 Screen Specifications

[Specify the 5--6 most important screens: design image, controls, events and validation. For the other screens, the design image is enough.]

6.2.1 Screen ``A''

#	Control	Type	Description / validation	Event handling

6.3 Accessibility Checklist

- Text contrast $\geq 4.5{:}1$; colour is not the only way to convey information
- All functions usable with the keyboard; visible focus
- Form fields have labels; error messages explain how to fix the problem
- Layout works on mobile and desktop (responsive)

7 Traceability Update

[Extend the traceability matrix from PA2 with the design elements (component, class, screen) that realise each requirement.]

8 Changes to the Requirements Since PA2

[List any changes made to the PA2 requirements after submission: what changed, why, and which items were affected.]

#	Change	Reason	Affected items

9 AI Usage Log

[Required in every PA (course GenAI policy). List each significant use of AI tools. Write ``No AI tools were used'' if none. Undisclosed AI use is treated as an academic integrity violation.]

#	Member	Tool	Task / purpose	How the output was verified and changed
1	Phụ trách mục 4.2--4.6	Codex	Hỗ trợ soạn OOD: class diagram, đặc tả 8 lớp, 5 sơ đồ sequence cho 5 use case, 3 state machine và bảng nguyên tắc thiết kế.	Đối chiếu PA3 section 2/4.1 và UC của PA2; kiểm tra tên operation, tham chiếu và build XeLaTeX, rà soát PDF. Các quyết định chi tiết hóa được ghi ở đầu 4.2; chưa kiểm thử ứng dụng hoặc xác nhận review của nhóm.
